

MATLAB[®] **para ingenieros**

Holly Moore



Ambiente MATLAB

Objetivos

Después de leer este capítulo, el alumno será capaz de

- iniciar el programa MATLAB y resolver problemas simples en la ventana de comando.
- comprender el uso de matrices que hace MATLAB.
- identificar y usar las diversas ventanas de MATLAB.
- definir y usar matrices simples.
- nombrar y usar variables.
- entender el orden de operaciones en MATLAB.
- comprender la diferencia entre cálculos escalares, de arreglo y matriciales en MATLAB.
- expresar números en notación de punto flotante y científica.
- ajustar el formato que se usa para desplegar números en la ventana de comandos.
- guardar el valor de las variables que se usen en una sesión de MATLAB.
- guardar una serie de comandos en un archivo-m.

2.1 INICIO

Usar MATLAB por primera vez es fácil; dominarlo puede tomar años. En este capítulo se introducirá al lector al ambiente de MATLAB y se le mostrará cómo efectuar cálculos matemáticos básicos. Después de leer este capítulo, será capaz de empezar a usar MATLAB para hacer sus tareas o en el trabajo. Por supuesto, conforme complete el resto de los capítulos podrá hacer más cosas.

Dado que el procedimiento para instalar MATLAB depende de su sistema operativo y del ambiente de la computadora, se supondrá que el lector ya instaló MATLAB en su computadora o que trabaja en un laboratorio de computación donde ya se instaló MATLAB. Para iniciar MATLAB, ya sea en el ambiente Windows o en el de Apple, haga clic en el icono del escritorio, o use el menú inicio para encontrar el programa. En el ambiente UNIX, escriba **Matlab** en el shell prompt (línea de comandos). No importa cómo lo inicie, una vez abierto MATLAB debe ver el prompt (incitador) de MATLAB (**>>** o **EDU>>**), que le indica que MATLAB está listo para que ingrese un comando. Cuando termine su sesión con MATLAB, puede salir del programa al escribir **quit** o **exit** en el prompt de MATLAB. MATLAB también utiliza la barra de menú estándar de Windows, de modo que puede salir del programa si elige **EXIT MATLAB** en el menú File (Archivo) o al seleccionar el icono de cerrar (x) en la esquina superior derecha de la pantalla. En la figura 2.1 se muestra por defecto la pantalla de MATLAB, la cual se abre siempre que inicia el programa.

Para comenzar a usar MATLAB, sólo necesita prestar atención a la ventana de comandos (a la derecha de la pantalla). En ella puede realizar cálculos en forma similar a como lo hace en una calculadora científica. Incluso, la mayoría de la sintaxis es la misma. Por ejemplo, para calcular el valor de 5 al cuadrado, se escribe el comando

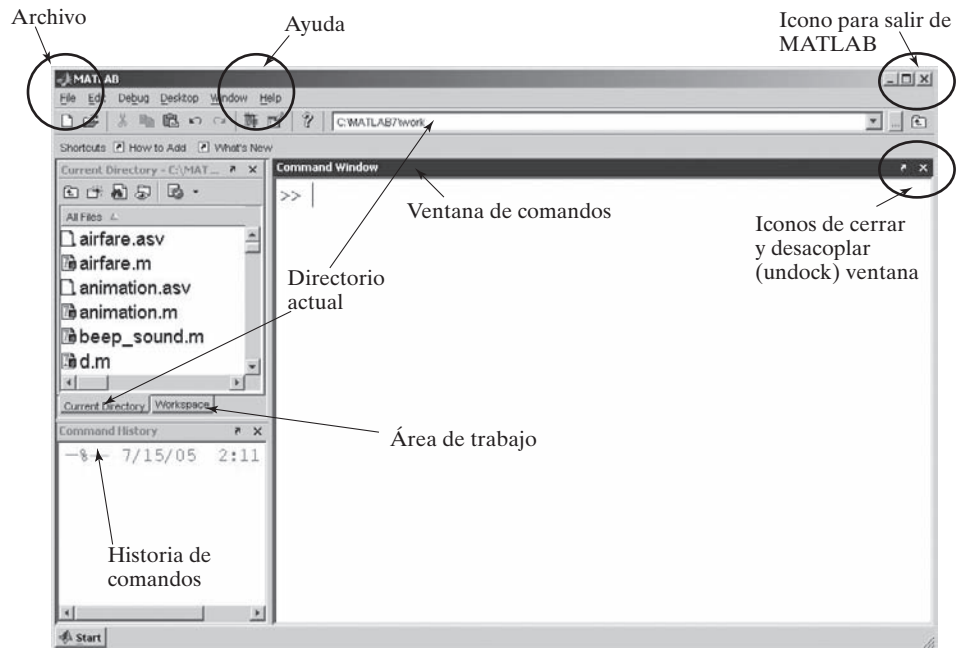
```
5^2
```

Se desplegará la salida siguiente:

```
ans =
```

Figura 2.1

Ventana de apertura de MATLAB. El ambiente de MATLAB consta de algunas ventanas, cuatro de las cuales se abren en la vista por defecto. Otras se abren conforme se necesiten durante una sesión de MATLAB.



O bien, para encontrar el valor de $\cos(\pi)$, escriba

```
cos(pi)
```

que da como resultado la salida

```
ans =  
-1
```

Idea clave: MATLAB usa las reglas algebraicas estándar para el orden de operación.

MATLAB usa las reglas algebraicas estándar para ordenar operaciones, lo que se vuelve importante cuando encadena cálculos. Estas reglas se estudian en la sección 2.3.2.

Sugerencia

Es posible que piense que algunos de los ejemplos son demasiado sencillos como para escribirlos usted mismo, que es suficiente con leer el material. Sin embargo, ¡recordará mejor el material si lo lee y lo escribe!

Antes de continuar, intente resolver el ejercicio de práctica 2.1.

Ejercicio de práctica 2.1

Escriba las siguientes expresiones en el prompt de comando de MATLAB y observe los resultados:

1. $5+2$
2. $5*2$
3. $5/2$

4. $3 + 2 * (4 + 3)$
5. $2.54 * 8 / 2.6$
6. $6.3 - 2.1045$
7. 3.6^2
8. $1 + 2^2$
9. `sqrt(5)`
10. `cos(pi)`

Sugerencia

Es posible que encuentre frustrante descubrir que, cuando comete un error, no se puede volver a escribir el comando después de haberlo ejecutado. Esto ocurre porque la ventana de comandos crea una lista de todos los comandos que ingresó. No se puede “desejecutar” o “descrear” un comando. Lo que puede hacer es introducir el comando en forma correcta y luego ejecutar esta nueva versión. MATLAB le ofrece varias maneras de facilitar lo anterior. Una de ellas es usar las teclas de flecha, que, por lo general, se encuentran en el lado derecho del teclado. La tecla hacia arriba, $\uparrow$, le permite desplazarse a través de la lista de comandos ejecutados. Una vez que encuentre el comando apropiado, puede editarlo y luego ejecutar la versión nueva. Esto ahorra tiempo en verdad. Sin embargo, también siempre es posible sólo volver a escribir el comando.

2.2 VENTANAS DE MATLAB

MATLAB utiliza varias ventanas de despliegue. La vista por defecto, que se presenta en la figura 2.1, incluye una gran **command window** (ventana de comandos) a la derecha y, apiladas a la izquierda, se encuentran las ventanas **current directory** (directorio actual), **workspace** (área de trabajo) y **command history** (historia de comandos). Observe las pestañas abajo a la izquierda de las ventanas; dichas pestañas le permiten acceder a las ventanas ocultas. Las versiones antiguas de MATLAB también incluían una ventana de **launch pad** (lanzamiento), que se reemplazó con el botón de **start** (inicio) en la esquina inferior izquierda. Además, cuando sea necesario, se abrirán automáticamente ventanas de **document** (documento), **graphics** (gráficas) y **editing** (edición). Cada una de esas ventanas se describe en las secciones que siguen. MATLAB también tiene construida internamente una función de ayuda, a la que se puede acceder desde la barra de menú, como se muestra en la figura 2.1. Para personalizar el escritorio, puede redimensionar cualquiera de estas ventanas, cerrar las que no use con el icono de cerrar (la x en la esquina superior derecha de cada ventana), o “desacoplarlas” con el icono undock (desacoplar), $\square$, que también se localiza en la esquina superior derecha de cada ventana.

2.2.1 Ventana de comandos (command window)

La ventana de comandos se localiza en el lado derecho de la vista por defecto de la pantalla de MATLAB, como se muestra en la figura 2.1. La ventana de comandos ofrece un ambiente similar a una memoria de trabajo auxiliar (scratch pad). El empleo de la ventana de comandos le permite guardar los valores que calcule, mas no los **comandos** que usó para generarlos. Si desea guardar la secuencia de comandos, necesitará emplear la ventana de edición para crear un **archivo-m** (m-file). Los archivos-m se describen en la sección 2.4.2. Ambos enfoques son valiosos; sin embargo, primero se enfatizará el uso de la ventana de comandos, antes de introducir los archivos-m.

Idea clave: la ventana de comandos es similar a una memoria de trabajo auxiliar.

Idea clave: la historia de comandos registra todos los comandos que se escribieron en la ventana de comandos.

2.2.2 Historia de comandos (command history)

La ventana de *historia de comandos* registra los comandos que se escriben en la ventana de comandos. Cuando sale de MATLAB, o cuando escribe el comando **clc**, la ventana de comandos se limpia (clear). Sin embargo, la ventana de historia de comandos conserva una lista de todos sus comandos. También puede limpiar la historia de comandos con el menú *edit*. Si trabaja en una computadora pública, entonces, como medida de seguridad, las opciones de MATLAB por defecto se pueden establecer de modo que limpie la historia cuando salga del programa. Si introdujo los comandos de muestra anteriores, observará que se repiten en la ventana de historia de comandos. Esta ventana es valiosa por varias razones, dos de las cuales son: porque permite revisar sesiones anteriores de MATLAB y porque se puede usar para transferir comandos a la ventana de comandos. Por ejemplo, primero limpie el contenido de la ventana de comandos al escribir

clc

Esta acción limpia la ventana de comandos, pero deja intactos los datos de la ventana de historia de comandos. Usted puede transferir cualquier comando desde la ventana de historia de comandos hacia la ventana de comandos al hacer doble clic (lo que también ejecuta el comando) o al hacer clic y arrastrar la línea de código a la ventana de comandos. Intente hacer doble clic

cos(pi)

en la ventana de historia de comandos. Debe regresar

ans =
-1

Ahora haga clic y arrastre

5^2

desde la ventana de historia de comando hacia la ventana de comandos. El comando no se ejecutará hasta que oprima *enter*, y entonces obtendrá el resultado:

ans =
25

Conforme ejecute cálculos cada vez más complicados en la ventana de comandos, encontrará que la ventana de historia de comandos es útil.

2.2.3 Ventana del área de trabajo (workspace)

Idea clave: la ventana workspace (área de trabajo) lista la información que describe todas las variables que crea el programa.

La *ventana del área de trabajo* le mantiene informado de las *variables* que usted define conforme ejecuta comandos en la ventana de comandos. Si ha hecho los ejemplos, la ventana del área de trabajo debe mostrar sólo una variable, **ans**, y decir que tiene un valor de 25 y que es un arreglo doble:

Name	Value	Class
 ans	25	double array

Haga que la ventana del área de trabajo diga algo más acerca de esta variable al hacer clic con el botón derecho sobre la barra con las etiquetas de las columnas. (Esta característica es nueva en MATLAB 7 y no funcionará con una versión anterior.) Revise **size** (tamaño) y **bytes**, además de **name** (nombre), **value** (valor) y **class** (clase). La ventana del área de trabajo ahora debe mostrar la siguiente información:

Name	Value	Size	Bytes	Class
 ans	25	1 × 1	8	double array

El símbolo en forma de retícula indica que la variable **ans** es un arreglo. El tamaño, 1×1 , dice que es un solo valor (una fila por una columna) y, por tanto, es un escalar. El arreglo usa 8 bytes de memoria. MATLAB está escrito en lenguaje C, y la designación de clase dice que, en lenguaje C, **ans** es un arreglo de punto flotante y doble precisión. En este momento, basta saber que la variable **ans** puede almacenar un número punto flotante (un número con punto decimal). En realidad, MATLAB considera a todo número que se ingrese como si fuera punto flotante, se escriban decimales o no.

Es posible definir variables adicionales en la ventana de comandos, y se listarán en la ventana del área de trabajo. Por ejemplo, al escribir

```
A = 5
```

regresa

```
A =
    5
```

Observe que la variable **A** se agregó a la ventana del área de trabajo, que lista las variables en orden alfabético. Las variables que comienzan con letras mayúsculas se listan en primer lugar, seguidas por las variables que comienzan con letras minúsculas.

Name	Value	Size	Bytes	Class
 A	5	1 × 1	8	double array
 ans	25	1 × 1	8	double array

En la sección 2.3.2 se estudiará en detalle cómo introducir matrices a MATLAB. Por ahora, usted puede ingresar una matriz unidimensional sencilla al escribir

```
B = [1, 2, 3, 4]
```

Este comando regresa

```
B =
    1     2     3     4
```

Las comas son opcionales; se obtendría el mismo resultado con

```
B = [ 1  2  3  4]
B =
    1     2     3     4
```

Observe que la variable **B** se agregó a la ventana del área de trabajo y que su tamaño es un arreglo 1×4 :

Name	Value	Size	Bytes	Class
 A	5	1 × 1	8	double array
 B	[1 2 3 4]	1 × 4	32	double array
 ans	25	1 × 1	8	double array

Idea clave: el tipo de datos por defecto es de números punto flotante y doble precisión almacenados en una matriz.

En forma similar se definen matrices bidimensionales. Se emplea punto y coma para separar las filas. Por ejemplo:

```
C = [ 1 2 3 4; 10 20 30 40; 5 10 15 20]
```

regresa

```
C =  
    1     2     3     4  
   10    20    30    40  
    5    10    15    20
```

Name	Value	Size	Bytes	Class
 A	5	1 × 1	8	double array
 B	[1 2 3 4]	1 × 4	32	double array
 C	<3 x 4 double>	3 × 4	96	double array
 ans	25	1 × 1	8	double array

Observe que **C** aparece en la ventana del área de trabajo como una matriz de 3 × 4. Para conservar espacio, no se mencionan los valores almacenados en la matriz.

Se pueden recuperar los valores para cualquier variable al escribir en el nombre de la variable. Por ejemplo, al ingresar

```
A
```

se obtiene

```
A =  
    5
```

Aunque las únicas variables que se introdujeron son matrices que contienen números, son posibles otros tipos de variables.

Al describir la ventana de comandos se introdujo el comando **clc**. Este comando limpia la ventana de comandos y deja una página en blanco para que usted trabaje en ella. Sin embargo, no borra de la memoria las variables reales que creó. El comando **clear** (limpiar) borra todas la variables guardadas. La acción del comando **clear** se refleja en la ventana del área de trabajo. Inténtelo al escribir

```
clear
```

en la ventana de comandos. La ventana del área de trabajo ahora está vacía:

Name	Value	Size	Bytes	Class
------	-------	------	-------	-------

Si usted suprime la ventana del área de trabajo (al cerrarla o desde el menú file o con el icono de cierre en la esquina superior derecha de la ventana), todavía podrá descubrir cuáles variables se definieron mediante el comando **whos**:

```
whos
```

Si se hubiese ejecutado antes de ingresar el comando **clear**, **whos** habría regresado

Name	Size	Bytes	Class
A	1x1	8	double array
B	1x4	32	double array
C	3x4	96	double array
ans	1x1	8	double array

Grand total is 18 elements using 144 bytes

2.2.4 Ventana de directorio actual (current directory)

La ventana de directorio actual lista todos los archivos en una carpeta de la computadora llamada directorio actual. Cuando MATLAB ingresa a archivos o guarda información, usa el directorio actual a menos que se diga algo diferente. La ubicación por defecto del directorio actual varía con su versión del software y con cómo se instaló. Sin embargo, el directorio actual se cita en la parte superior de la ventana principal. El directorio actual se puede cambiar al seleccionar otro directorio de la lista desplegable que se ubica junto a la lista de directorio o al navegar entre los archivos de su computadora. La navegación se lleva a cabo con el botón browse, que se ubica junto a la lista desplegable. (Véase la figura 2.2.)

2.2.5 Ventana de documento (document window)

Hacer doble clic sobre cualquier variable mencionada en la ventana del área de trabajo lanza automáticamente una ventana de documento que contiene el **array editor** (editor de arreglos). Los valores que se almacenan en la variable se despliegan en un formato de hoja de cálculo. Puede cambiar los valores en el editor de arreglos o puede agregar nuevos valores. Por ejemplo, si todavía no ingresa la matriz bidimensional C, ingrese el siguiente comando en la ventana de comandos:

```
C = [ 1 2 3 4; 10 20 30 40; 5 10 15 20];
```

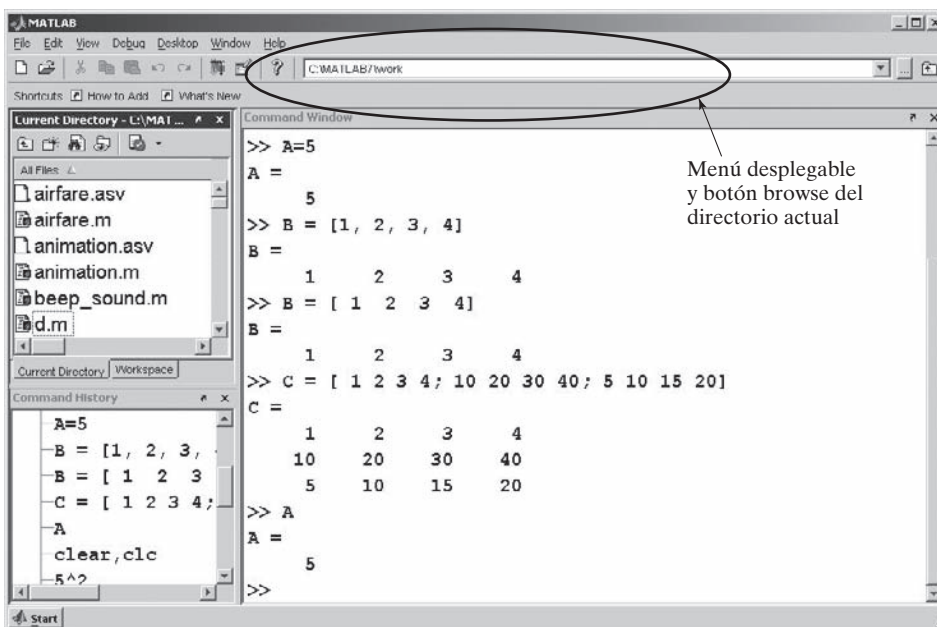


Figura 2.2

La **ventana de directorio actual** lista todos los archivos en el directorio actual. Puede cambiar el directorio actual o en el menú desplegable o con el botón browse.

Icono de
variable
nueva

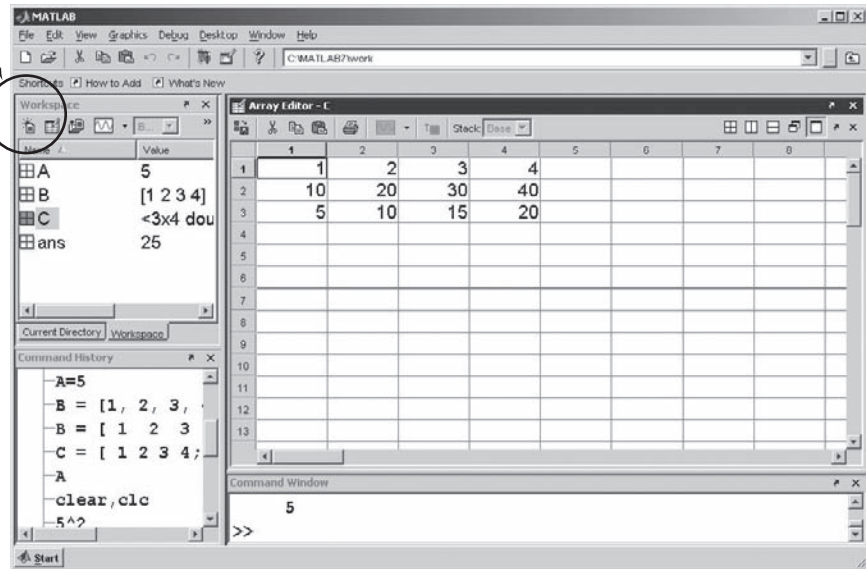


Figura 2.3

La **ventana de documentos** despliega el **editor de arreglos**.

Idea clave: un punto y coma suprime la salida de los comandos escritos en la ventana de comandos.

Poner punto y coma al final del comando suprime la salida, de modo que no se repita en la ventana de comandos. Sin embargo, ahora **C** se debe citar en la ventana del área de trabajo. Haga doble clic en ella. Sobre la ventana de comandos se abrirá una ventana de documento, como se muestra en la figura 2.3. Ahora puede agregar más valores a la matriz **C** o cambiar los valores existentes.

La ventana de documento/editor de arreglos también se puede usar en conjunto con la ventana del área de trabajo para crear arreglos completamente nuevos. Corra su ratón lentamente sobre los iconos en la barra de atajos en lo alto de la ventana del área de trabajo. Si es paciente, deberá aparecer la función de cada icono. El icono de variable nueva se parece a una página con un gran asterisco detrás suyo. Seleccione el icono de variable nueva y, en la lista de variables, deberá aparecer una nueva variable llamada **unnamed** (sin nombrar). Puede cambiar su nombre al hacer clic derecho y seleccionar **rename** (renombrar) del menú secundario. Para agregar valores a esta variable nueva, haga doble clic sobre ella y agregue sus datos desde la ventana de editor de arreglo. El botón de variable nueva es una nueva característica de MATLAB 7; si utiliza una versión anterior, no podrá crear variables de esta forma.

Cuando termine de crear variables nuevas, cierre el editor de arreglos al seleccionar el icono de cerrar ventana en la esquina superior derecha de la ventana.

2.2.6 Ventana gráficas (graphics window)

La ventana de gráficas se lanza automáticamente cuando solicita una gráfica. Para demostrar esta característica, primero cree un arreglo de valores **x**:

```
x = [ 1 2 3 4 5];
```

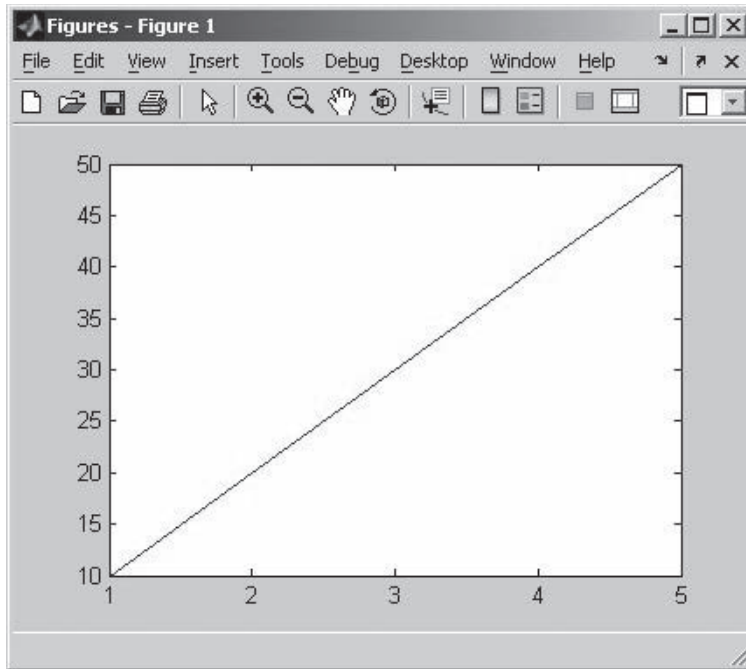
(Recuerde que el punto y coma suprime la salida de este comando; sin embargo, en la ventana del área de trabajo aparece una variable nueva, **x**.)

Ahora cree una lista de valores **y**:

```
y = [10 20 30 40 50];
```

Para crear una gráfica, use el comando plot:

```
plot(x,y)
```

**Figura 2.4**

MATLAB hace fácil la creación de gráficas.

La ventana de gráficas se abre automáticamente. (Véase la figura 2.4.) Note que en la barra de tareas aparece una nueva etiqueta de ventana al fondo de la pantalla de la ventana. Se titulará **<Student Version> Figure...** o simplemente **Figure 1**, dependiendo de si usa la versión estudiantil o profesional, respectivamente, del software. Cualesquiera gráficas adicionales que cree sobrescribirán la figura 1 a menos que ordene específicamente a MATLAB que abra una nueva ventana de gráficas.

MATLAB facilita la modificación de las gráficas al agregar títulos, etiquetas x y y , líneas múltiples, etc. ¡Los ingenieros y científicos **nunca** presentan una gráfica sin etiquetas!

Idea clave: agregue siempre un título y etiquetas de ejes a las gráficas.

2.2.7 Ventana de edición (edit window)

La ventana de edición se abre al elegir **File** de la barra de menú, luego **New** y, finalmente, **M-file (File → New → M-file)**. Esta ventana le permite escribir y guardar una serie de comandos sin ejecutarlos. También puede abrir la ventana de edición al escribir **edit** en el prompt de comando o al elegir el botón New File (archivo nuevo) en la barra de herramientas (toolbar).

2.2.8 Botón de inicio

El botón de inicio se ubica en la esquina inferior izquierda de la ventana de MATLAB. Ofrece un acceso alternativo a las diversas ventanas de MATLAB, así como a la función de ayuda, productos de Internet y cajas de herramientas de MATLAB. Las cajas de herramientas proporcionan funcionalidad MATLAB adicional, para áreas de contenido específico. En particular, la caja de herramientas simbólica es enormemente útil a científicos e ingenieros. El botón de inicio es nuevo a MATLAB 7 y sustituye la ventana de lanzamiento que se usaba en MATLAB 6.

2.3 RESOLUCIÓN DE PROBLEMAS CON MATLAB

El ambiente de ventana de comandos es una poderosa herramienta para resolver problemas de ingeniería. Para usarla de manera efectiva, necesitará entender más acerca de cómo funciona MATLAB.

2.3.1 Uso de variables

Aunque es posible resolver muchos problemas al usar MATLAB como calculadora, usualmente es más conveniente dar nombres a los valores que utiliza. MATLAB usa las convenciones de nomenclatura comunes a la mayoría de los programas de cómputo:

- Todos los nombres deben comenzar con una letra. Los nombres pueden tener cualquier longitud, pero en MATLAB 7 sólo se usan los primeros 63 caracteres. (Use el comando **namelengthmax** para confirmar esto cuando instale MATLAB.) Aunque MATLAB le permitirá crear nombres de variable largos, la longitud excesiva crea una significativa oportunidad de error. Un lineamiento común es usar letras minúsculas y números en los nombres de variable y usar letras mayúsculas para los nombres de constantes. Sin embargo, si una constante tradicionalmente se expresa como una letra minúscula, siéntase en libertad de seguir dicha convención. Por ejemplo, en los textos de física, la rapidez de la luz siempre es la letra *c* minúscula. Los nombres deben ser lo suficientemente cortos como para recordarlos y que sean descriptivos.
- Los únicos caracteres permisibles son letras, números y el guión bajo. Con el comando **isvarname** puede verificar si se permite el nombre de la variable. Como es estándar en los lenguajes de computación, el número 1 significa que algo es verdadero y el número 0 significa falso. En consecuencia,

```
isvarname time
ans =
    1
```

indica que **time** es un nombre de variable legítimo, y

```
isvarname cool-beans
ans =
    0
```

dice que **cool-beans** no es un nombre de variable legítimo.

- Los nombres son sensibles a mayúsculas/minúsculas. La variable **x** es diferente de la variable **X**.
- MATLAB reserva una lista de palabras clave para uso del programa, que no se pueden asignar como nombres de variable. El comando **iskeyword** hace que MATLAB elabore una lista de tales nombres reservados:

```
iskeyword
ans =
    'break'
    'case'
    'catch'
    'continue'
    'else'
    'elseif'
    'end'
    'for'
    'function'
    'global'
    'if'
    'otherwise'
    'persistent'
    'return'
    'switch'
    'try'
    'while'
```

- MATLAB le permite reasignar nombres de función internos como nombres de variable. Por ejemplo, podría crear una nueva variable llamada **sin** con el comando

```
sin=4
```

que regresa

```
sin =  
4
```

Esto es claramente una práctica peligrosa, pues la función **sin** (es decir: seno) ya no está disponible. Si intenta usar la función sobrescrita, obtendrá un enunciado de error (“Índice supera dimensiones de matriz”):

```
sin(3)  
??? Index exceeds matrix dimensions.
```

Puede verificar si una variable es una función MATLAB interna al usar el comando **which**:

```
which sin  
sin is a variable.
```

Puede restablecer **sin** a una función al escribir

```
clear sin
```

Ahora, cuando pregunte

```
which sin
```

la respuesta es

```
C:\MATLAB7\toolbox\matlab\elfun\sin.m
```

que menciona la ubicación de la función interna.

Ejercicio de práctica 2.2

¿Cuál de los siguientes nombres se permiten en MATLAB? Haga sus predicciones y luego pruébelas con los comandos **isvarname**, **iskeyword** y **which**.

1. test
2. Test
3. if
4. mi-libro
5. mi_libro
6. Esteesunnombremuylargoperoinclusoasispermite?
7. 1ergrupo
8. grupo_uno
9. zzaAbc
10. z34wAwy?12#
11. sin
12. log

Idea clave: la matriz es el tipo de datos principal en MATLAB y puede retener información numérica así como otros tipos de información.

vector: matriz compuesta de una sola fila o una sola columna

escalar: matriz de un solo valor

2.3.2 Matrices en MATLAB

El tipo de datos básico que se usa en MATLAB es la *matriz*. Un solo valor, llamado *escalar*, se representa como una matriz 1×1 . Una lista de valores, ordenados o en una columna o en una fila, es una matriz unidimensional que se llama *vector*. Una tabla de valores se representa como una matriz bidimensional. Aunque este capítulo se limitará a escalares, vectores y matrices, MATLAB puede manejar arreglos de orden superior.

En nomenclatura matemática, las matrices se representan como filas y columnas dentro de corchetes:

$$A = [5] \quad B = [2 \quad 5] \quad C = \begin{bmatrix} 1 & 2 \\ 5 & 7 \end{bmatrix}$$

En este ejemplo, *A* es una matriz 1×1 , *B* es una matriz 1×2 y *C* es una matriz 2×2 . La ventaja de usar representación matricial es que todos los grupos de información se pueden representar con un solo nombre. La mayoría de personas se sienten más cómodas al asignar un nombre a un solo valor, así que se comenzará por explicar cómo MATLAB maneja los escalares y luego se avanzará a matrices más complicadas.

Operaciones escalares

MATLAB maneja operaciones aritméticas entre dos escalares en forma muy parecida a como lo hacen otros programas de cómputo e incluso su calculadora. En la tabla 2.1 se muestra la sintaxis para la suma, resta, multiplicación, división y exponenciación. El comando

$$a = 1 + 2$$

se debe leer como “a **a** se le asigna un valor de 1 más 2”, que es la suma de dos cantidades escalares. Las operaciones aritméticas entre dos variables escalares usa la misma sintaxis. Por ejemplo, suponga que usted definió **a** en el enunciado anterior y que **b** tiene un valor de 5:

$$b = 5$$

Entonces

$$x = a + b$$

regresa el siguiente resultado:

$$x = 8$$

Tabla 2.1 Operaciones aritméticas entre dos escalares (operaciones binarias)

Operación	Sintaxis algebraica	Sintaxis MATLAB
Suma	$a + b$	a + b
Resta	$a - b$	a - b
Multiplicación	$a \times b$	a * b
División	$\frac{a}{b}$ o $a \div b$	a / b
Exponenciación	a^b	a ^ b

En MATLAB, un solo signo igual (=) se llama operador asignación. El operador asignación hace que el resultado de sus cálculos se almacenen en una ubicación de memoria de la computadora. En el ejemplo anterior, a **x** se le asigna un valor de 8. Si usted ingresa el nombre de variable

x

en MATLAB, obtiene el siguiente resultado:

x =
8

El operador asignación es significativamente diferente de una igualdad. Considere el enunciado

x = x + 1

Éste no es un enunciado algebraico válido, pues claramente **x** no es igual a **x + 1**. Sin embargo, cuando se interpreta como un enunciado de asignación, dice que se debe sustituir el valor actual de **x** almacenado en memoria con un nuevo valor que es igual a la antigua **x** más **1**.

Dado que el valor almacenado en **x** originalmente fue 8, el enunciado regresa

x =
9

lo que indica que el valor almacenado en la ubicación de memoria llamada **x** cambió a 9. El enunciado de asignación es similar al proceso familiar de guardar un archivo. Cuando guarda por primera vez un documento en un procesador de palabras, usted le asigna un nombre. Subsecuentemente, después de que realiza cambios, vuelve a guardar su archivo, pero todavía le asigna el mismo nombre. Las versiones primera y segunda no son iguales: sólo asignó una nueva versión de su documento a una ubicación de memoria existente.

Orden de las operaciones

En todos los cálculos matemáticos es importante entender el orden en el que se realizan las operaciones. MATLAB sigue las reglas algebraicas estándar para el orden de operación:

- Primero realiza los cálculos adentro de paréntesis, desde el conjunto más interno hasta el más externo.
- A continuación, realiza operaciones de exponenciación.
- Luego realiza operaciones de multiplicación y división de izquierda a derecha.
- Finalmente, realiza operaciones de suma y resta de izquierda a derecha.

Para entender mejor la importancia del orden de las operaciones, considere los cálculos que se involucran al encontrar el área de un cilindro circular recto.

El área es la suma de las áreas de las dos bases circulares y el área de la superficie curva entre ellas, como se muestra en la figura 2.5. Si la altura (height) del cilindro es 10 cm y el radio (radius) es de 5 cm, se puede usar el siguiente código MATLAB para encontrar el área (surface_area):

```
radius = 5;
height = 10;
surface_area = 2*pi*radius^2 + 2*pi*radius*height
```

El código regresa

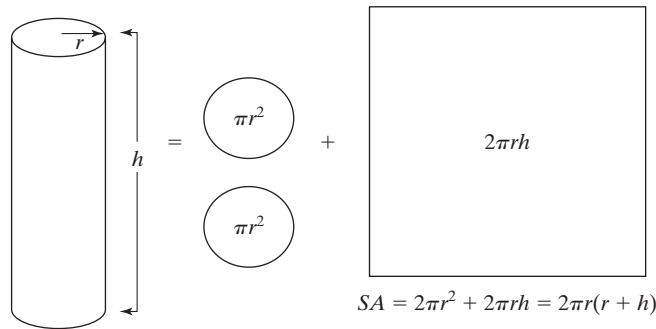
surface_area =
471.2389

En este caso, MATLAB realiza primero la exponenciación y eleva el radio a la segunda potencia. Luego trabaja de izquierda a derecha y calcula el primer producto y luego el

Idea clave: el operador asignación es diferente de una igualdad.

Figura 2.5

Encontrar el área de un cilindro circular recto involucra suma, multiplicación y exponenciación.



segundo producto. Finalmente, suma los dos productos. En vez de ello, podría formular la expresión como

```
surface_area = 2*pi*radius*(radius + height)
```

que también regresa

```
surface_area =  
471.2389
```

En este caso, MATLAB primero encuentra la suma del radio y la altura y luego realiza las multiplicaciones, trabajando de izquierda a derecha. Si olvida incluir los paréntesis, tendría

```
surface_area = 2*pi*radius*radius + height
```

en cuyo caso el programa primero tendría que calcular el producto de **2*pi*radius*radius** y luego sumar **height**, lo que obviamente resulta en la respuesta incorrecta. Note que fue necesario incluir el operador multiplicación antes de los paréntesis, porque MATLAB no supone operador alguno e interpretaría la expresión

```
radius(radius + height)
```

como la función **radius** con entrada **(radius + height)**. Puesto que no hay función **radio** en MATLAB, esta interpretación resultaría en un enunciado de error.

Es importante tener mucho cuidado al convertir ecuaciones en enunciados MATLAB. No hay penalización por agregar paréntesis adicionales y con frecuencia esto hace al código más fácil de interpretar, tanto para el programador como para otros que puedan usar el código en el futuro.

Otra forma de hacer más legible un código de cómputo es descomponer las expresiones largas en múltiples enunciados. Por ejemplo, considere la ecuación

$$f = \frac{\log(ax^2 + bx + c) - \sin(ax^2 + bx + c)}{4\pi x^2 + \cos(x - 2) * (ax^2 + bx + c)}$$

Sería muy fácil cometer un error de tecla en esta ecuación. Para minimizar la posibilidad de que ello ocurra, descomponga la ecuación en muchas piezas. Por ejemplo, primero asigne valores para **x**, **a**, **b** y **c**:

```
x = 9;  
a=1;  
b=3;  
c=5;
```

Luego defina un polinomio y el denominador:

```
poly = a*x^2 + b*x + c;  
denom = 4*pi*x^2 + cos(x - 2)*poly;
```

Combine estos componentes en una ecuación final:

$$f = (\log(\text{poly}) - \sin(\text{poly})) / \text{denom}$$

El resultado es

$$f = 0.0044$$

Como se mencionó, este enfoque minimiza su oportunidad de error. En lugar de teclear el polinomio tres veces (y arriesgarse a un error cada vez), sólo necesita teclear una sola vez. Aumenta la probabilidad de crear código MATLAB preciso y es más fácil que otros lo entiendan.

Idea clave: intente minimizar su oportunidad de error.

Sugerencia

MATLAB no lee “espacio blanco”, así que no importa si agrega espacios a sus comandos. Es más fácil leer una expresión larga si agrega un espacio antes y después de los signos más (+) y menos (−), pero no antes y después de los signos de multiplicación (*) y división (/).

Ejercicio de práctica 2.3

Prediga los resultados de las siguientes expresiones MATLAB y luego verifique sus predicciones al teclear las expresiones en la ventana de comandos:

1. $6/6 + 5$
2. $2 * 6^2$
3. $(3+5) * 2$
4. $3 + 5 * 2$
5. $4 * 3/2 * 8$
6. $3 - 2/4 + 6^2$
7. $2^3 * 4$
8. $2^3 * 4$
9. $3^5 + 2$
10. $3^5 + 2$

Cree y pruebe la sintaxis MATLAB para evaluar las siguientes expresiones y luego verifique sus respuestas con una calculadora de mano.

11. $\frac{5 + 3}{9 - 1}$
12. $2^3 - \frac{4}{5 + 3}$
13. $\frac{5^{2+1}}{4 - 1}$
14. $4 \frac{1}{2} * 5 \frac{2}{3}$
15. $\frac{5 + 6 * \frac{7}{3} - 2^2}{\frac{2}{3} * \frac{3}{3 * 6}}$

EJEMPLO 2.1**Operaciones escalares**

Los túneles de viento (véase la figura 2.6) juegan un importante papel en la comprensión del comportamiento de las aeronaves de alto rendimiento. Con la finalidad de interpretar los datos del túnel de viento, los ingenieros requieren entender cómo se comportan los gases. La ecuación básica que describe las propiedades de los gases es la ley del gas ideal, una relación que se estudia con detalle en las clases de química de primer año. La ley establece que

$$PV = nRT$$

donde P = presión en kPa,
 V = volumen en m^3 ,
 n = número de kmoles de gas en la muestra,
 R = constante de gas ideal, $8.314 \text{ kPa m}^3/\text{kmol K}$, y
 T = temperatura, en grados kelvin (K).

Además, se sabe que el número de kmoles de gas es igual a la masa del gas dividida por la masa molar (también conocida como peso molecular) o

$$n = m/MW$$

donde

m = masa en kg y
 MW = masa molar en kg/kmol .

En las ecuaciones se pueden usar diferentes unidades si el valor de R se cambia en concordancia.

Suponga ahora que usted sabe que el volumen de aire en el túnel de viento es de 1000 m^3 . Antes de que el túnel de viento se encienda, la temperatura del aire es de 300 K , y la presión es de 100 kPa . La masa molar (peso molecular) promedio del aire es aproximadamente 29 kg/kmol . Encuentre la masa del aire en el túnel de viento.

Para resolver este problema, use la siguiente metodología para resolución de problemas:

1. Establezca el problema.

Cuando usted resuelve un problema, es buena idea volver a enunciarlo en sus propias palabras: calcular la masa del aire en un túnel de viento.



Figura 2.6

Los túneles de viento se usan para probar el diseño de las aeronaves.
 (Cortesía de Louis Bencze/
 Stone/Getty Images Inc.)

2. Describa la entrada y la salida.

Entrada

Volumen	$V = 1000 \text{ m}^3$
Temperatura	$T = 300 \text{ K}$
Presión	$P = 100 \text{ kPa}$
Peso molecular	$MW = 29 \text{ kg/kmol}$
Constante de gas	$R = 8.314 \text{ kPa m}^3/\text{kmol K}$

Salida

Masa $m = ? \text{ Kg}$

3. Desarrolle un ejemplo a mano.

Trabajar el problema a mano (o con una calculadora) le permite subrayar un algoritmo, que usted puede traducir más tarde a código MATLAB. Debe elegir datos simples que hagan fácil la comprobación de su trabajo. En este problema, se conocen dos ecuaciones que relacionan los datos:

$$PV = nRT \quad \text{ley del gas ideal}$$

$$n = m/MW \quad \text{conversión de masa a moles}$$

Resuelva la ley del gas ideal para n y sustituya los valores dados:

$$n = Pv/RT$$

$$= (100 \text{ kPa} \times 1000 \text{ m}^3)/(8.314 \text{ kPa m}^3/\text{kmol K}) \times 300\text{K}$$

$$= 40.0930 \text{ kmol}$$

Convierta moles a masa al resolver la ecuación de conversión para la masa m y sustituya los valores:

$$m = n \times MW = 40.0930 \text{ kmol} \times 29 \text{ kg/kmol}$$

$$m = 1162.70 \text{ kg}$$

4. Desarrolle una solución MATLAB.

Primero, limpie la pantalla y la memoria:

clear, clc

Ahora realice los siguientes cálculos en la ventana de comandos:

```
P = 100
P =
    100
T = 300
T =
    300
V = 1000
V =
    1000
MW = 29
MW =
    29
R = 8.314
R =
    8.3140
```

```

n=(P*V)/(R*T)
n =
    40.0930
m = n*MW
m =
    1.1627e+003

```

Existen muchas cosas que usted debe notar acerca de esta solución MATLAB. Primero, puesto que no se utilizaron puntos y comas para suprimir la salida, los valores de las variables se repiten después de cada enunciado de asignación. Note también el uso de paréntesis en el cálculo de n . Son necesarios en el denominador, pero no en el numerador. Sin embargo, usar paréntesis en ambos hace al código más fácil de leer.

5. Ponga a prueba la solución.

En este caso, es suficiente comparar el resultado con el que se obtuvo a mano. Los problemas más complicados resueltos en MATLAB deben usar una diversidad de datos de entrada para confirmar que su solución funciona en una diversidad de casos. En la figura 2.7 se muestra la pantalla MATLAB que se usó para resolver este problema.

Note que las variables que se definieron en la ventana de comandos se citan en la ventana del área de trabajo. Note también que la historia de comandos lista los comandos que se ejecutaron en la ventana de comandos. Si se desplaza por la ventana de historia de comandos, verá comandos de sesiones previas de MATLAB. Todos estos comandos están disponibles para que usted los mueva a la ventana de comandos.

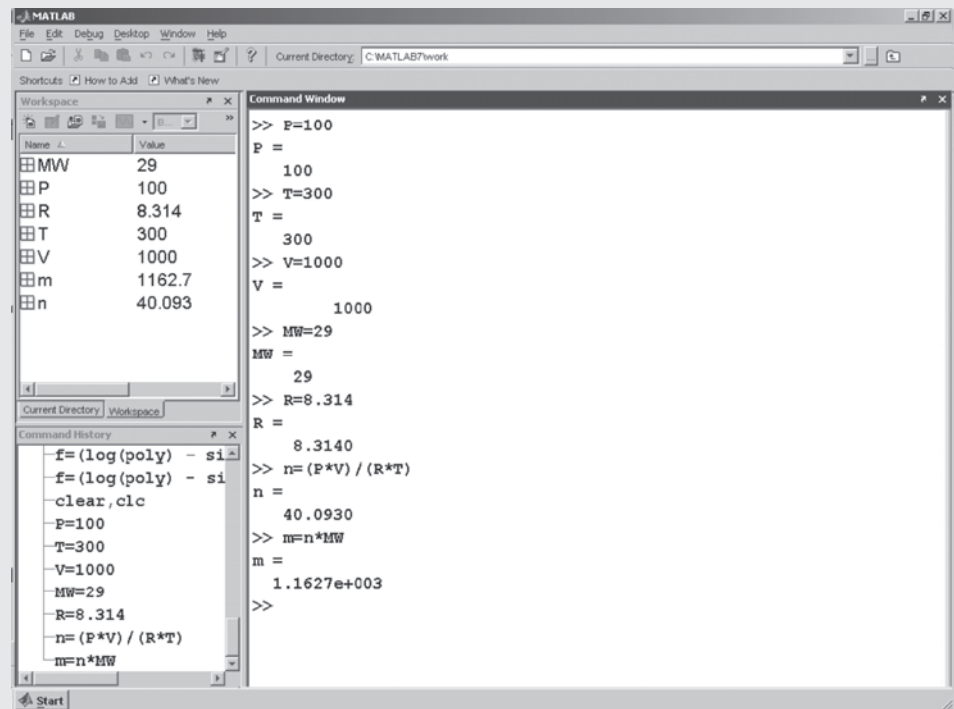


Figura 2.7

Pantalla de MATLAB que se usó para resolver el problema del gas ideal.

Operaciones de arreglos

Usar MATLAB como una calculadora glorificada está bien, pero su verdadera fortaleza está en las manipulaciones matriciales. Como se describió anteriormente, la forma más simple de definir una matriz es usar una lista de números, llamada *lista explícita*. El comando

```
x= [1 2 3 4]
```

regresa el vector fila

```
x =  
    1 2 3 4
```

Recuerde que, al definir este vector, puede hacer la lista de valores con o sin comas. Una nueva fila se indica mediante un punto y coma, de modo que un vector columna se especifica como

```
y= [ 1; 2; 3; 4]
```

y una matriz que contiene tanto filas como columnas se crearía con el enunciado

```
a = [ 1 2 3 4; 2 3 4 5 ; 3 4 5 6]
```

y regresaría

```
a =  
    1 2 3 4  
    2 3 4 5  
    3 4 5 6
```

Sugerencia

Es más fácil seguir la pista de cuántas variables ingresó en una matriz si ingresa cada fila en una línea separada:

```
a = [1 2 3 4;  
    2 3 4 5;  
    3 4 5 6]
```

Aunque una matriz complicada tiene que ingresarse a mano, las matrices con intervalos regulares se pueden ingresar mucho más fácilmente. El comando

```
b= 1:5
```

y el comando

```
b = [1:5]
```

regresan ambos una matriz fila

```
b =  
    1 2 3 4 5
```

(Los corchetes son opcionales.) El incremento por defecto es 1, pero si usted quiere usar un incremento diferente, colóquelo entre el primero y último valores en el lado derecho del comando. Por ejemplo,

```
c= 1:2:5
```

indica que el incremento entre los valores será 2 y regresa

```
c =  
    1    3    5
```

lista explícita: lista que identifica cada miembro de una matriz

Si usted quiere que MATLAB calcule el espaciado entre los elementos, puede usar el comando **linspace**. Especifique el valor inicial, el valor final y cuántos valores quiere en total. Por ejemplo,

```
d=linspace(1,10, 3)
```

regresa un vector con tres valores, espaciados igualmente entre 1 y 10:

```
d =  
1      5.5    10
```

Puede crear vectores espaciados logarítmicamente con el comando **logspace**, que también requiere tres entradas. Los primeros dos valores son potencias de 10 que representan los valores inicial y final en el arreglo. El valor final es el número de elementos en el arreglo. Por ende,

```
e=logspace(1,3,3)
```

regresa tres valores

```
e =  
10  100 1000
```

Note que el primer elemento en el vector es 10^1 y el último elemento en el arreglo es 10^3 .

Sugerencia

Puede incluir operaciones matemáticas dentro de un enunciado de definición de matriz. Por ejemplo, puede tener **a = [0: pi/10: pi]**.

Las matrices se pueden usar en muchos cálculos con escalares. Si **a = [1 2 3]** se puede sumar 5 a cada valor en la matriz con la sintaxis

```
b = a + 5
```

que regresa

```
b =  
6      7      8
```

Idea clave: la multiplicación de matrices es diferente de la multiplicación elemento por elemento.

Este enfoque funciona bien para suma y resta; sin embargo, la multiplicación y la división son un poco diferentes. En matemáticas matriciales, el operador de multiplicación (*) tiene un significado específico. Puesto que todas las operaciones MATLAB pueden involucrar matrices, es necesario un operador diferente para indicar multiplicación elemento por elemento. Dicho operador es **.*** (que se llama *multiplicación punto*). Por ejemplo,

```
a.*b
```

resulta en elemento 1 de la matriz **a** multiplicado por el elemento 1 de la matriz **b**, elemento 2 de la matriz **a** multiplicado por el elemento 2 de la matriz **b**, elemento *n* de la matriz **a** multiplicado por el elemento *n* de la matriz **b**.

Para el caso particular de **a** (que es [1 2 3]) y **b** (que es [6 7 8]),

```
a.*b
```

regresa

```
ans =  
6      14     24
```

(Haga las operaciones para convencerse de que son las respuestas correctas.)

Sólo usar `*` implica una multiplicación matricial, que en este caso regresaría un mensaje de error porque **a** y **b** no satisfacen aquí las reglas para multiplicación en álgebra matricial. La moraleja es: tenga cuidado al usar el operador correcto cuando quiera realizar multiplicación elemento por elemento (también llamado arreglo).

La misma sintaxis se cumple para la división elemento por elemento (`./`) y la exponenciación (`.^`) de elementos individuales:

```
a./b
a.^2
```

Como ejercicio, prediga los valores que resultan de las dos expresiones anteriores y luego pruebe sus predicciones mediante la ejecución de los comandos en MATLAB.

Ejercicio de práctica 2.4

Conforme realice los siguientes cálculos, recuerde la diferencia entre los operadores `*` y `.*`, así como los operadores `/` y `./` y entre `^` y `.^`:

1. Defina la matriz $a = [2.3 \ 5.8 \ 9]$ como una variable MATLAB.
2. Encuentre el seno de **a**.
3. Sume 3 a cada elemento en **a**.
4. Defina la matriz $b = [5.2 \ 3.14 \ 2]$ como una variable MATLAB.
5. Sume cada elemento de la matriz **a** y la matriz **b**.
6. Multiplique cada elemento en **a** por el correspondiente elemento en **b**.
7. Eleve al cuadrado cada elemento en la matriz **a**.
8. Cree una matriz llamada **c** de valores igualmente espaciados, desde 0 hasta 10, con un incremento de 1.
9. Cree una matriz llamada **d** de valores igualmente espaciados, desde 0 hasta 10, con un incremento de 2.
10. Use la función **linspace** para crear una matriz de seis valores igualmente espaciados, desde 10 hasta 20.
11. Use la función **logspace** para crear una matriz de cinco valores logarítmicamente espaciados entre 10 y 100.

La capacidad matricial de MATLAB hace fácil realizar cálculos repetitivos. Por ejemplo, suponga que tiene una lista de ángulos en grados que le gustaría convertir a radianes. Primero ponga los valores en una matriz. Para ángulos de 10, 15, 70 y 90, ingrese

```
degrees = [ 10 15 70 90];
```

Para cambiar los valores a radianes, debe multiplicar por $\pi/180$:

```
radians=degrees*pi/180
```

Este comando regresa una matriz llamada **radians**, con los valores en radianes. (¡Inténtelo!) En este caso, podría usar o el operador `*` o el `.*`, porque la multiplicación involucra una sola matriz (**degrees**) y dos escalares (π y 180). Por lo tanto, podría haber escrito

```
radians=degrees.*pi/180
```

Idea clave: la capacidad matricial de MATLAB hace sencillo realizar cálculos repetitivos.

Sugerencia

En MATLAB, el valor de π se construye como un número punto flotante llamado **pi**. Puesto que π es un número irracional, no se puede expresar *exactamente* con una representación en punto flotante, de modo que la constante **pi** de MATLAB en realidad es una aproximación. Puede ver esto cuando encuentra **sin(pi)**. De la trigonometría, la respuesta debería ser 0. Sin embargo, MATLAB regresa un número muy pequeño. El valor real depende de su versión del programa: la versión profesional 7 regresó 1.2246e-016. En la mayoría de los cálculos, esto no haría una diferencia en el resultado final.

Otro operador matricial útil es la transposición. El operador transpuesto cambia filas a columnas y viceversa. Por ejemplo,

```
degrees'
```

regresa

```
ans =
    10
    15
    70
    90
```

Esto facilita la creación de tablas. Por ejemplo, para crear una tabla que convierta grados a radianes, ingrese

```
table = [degrees' , radians']
```

que le pide a MATLAB crear una matriz llamada **table**, en la que la columna 1 es degrees (grados) y la columna 2 es radians (radianes):

```
table =
    10.0000    0.1745
    15.0000    0.2618
    70.0000    1.2217
    90.0000    1.5708
```

Si traspone una matriz bidimensional, todas las filas se convierten en columnas y todas las columnas se convierten en filas. Por ejemplo, el comando

```
table'
```

resulta en

```
    10.0000    15.0000    70.0000    90.0000
    0.1745    0.2618    1.2217    1.5708
```

Note que **table** no es un comando MATLAB, sino meramente un nombre de variable conveniente. Se podría haber usado cualquier nombre significativo, por decir, **conversiones** o **grados_a_radianes**.

EJEMPLO 2.2**Cálculos matriciales con escalares**

Los datos científicos, como los que se recolectan de los túneles de viento, usualmente están en unidades SI (Système International). Sin embargo, mucha de la infraestructura fabril de Estados Unidos se nominó en unidades inglesas (a veces llamadas Ingeniería Americana o Estándar Americano). Los ingenieros necesitan estar familiarizados con ambos sistemas y deben ser especialmente cuidadosos cuando comparten datos con otros ingenieros. Acaso el ejemplo más notorio de los problemas de confusión de unidades es el Mars Climate Orbiter (figura 2.8), que fue el segundo vuelo del programa Mars Surveyor. La nave espacial se quemó en la órbita de Marte, en septiembre de 1999, debido a una tabla de referencia incrustada en el software de la nave. La tabla, probablemente generada a partir de las pruebas en el túnel de viento, usaba libras fuerza (lbf) cuando el programa esperaba valores en newtons (N).

En este ejemplo, se usará MATLAB para crear una tabla de conversión de libras fuerza a newtons. La tabla comenzará en 0 e irá hasta 1000 lbf, en intervalos de 100 lbf. El factor de conversión es

$$1 \text{ lbf} = 4.4482216 \text{ N}$$

1. Establezca el problema.
Cree una tabla que convierta libras fuerza (lbf) a newtons (N).
2. Describa las entradas y salidas.

Entrada

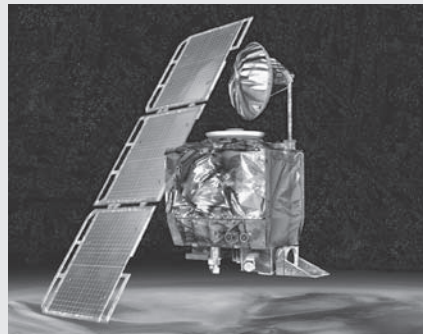
Valor inicial en la tabla	0 lbf
Valor final en la tabla	1000 lbf
Incremento entre valores	100 lbf
La conversión de lbf a N es	1 lbf = 4.4482216 N

Salida

Tabla que presenta libras fuerza (lbf) y newtons (N)

3. Desarrolle un ejemplo a mano.
Puesto que se creará una tabla, tiene sentido verificar algunos valores diferentes. Al elegir números para los que la matemática sea sencilla hace que el ejemplo a mano sea simple para completar, pero todavía valioso como comprobación:

0	*	4.4482216 = 0
100	*	4.4482216 = 444.82216
1000	*	4.4482216 = 4448.2216

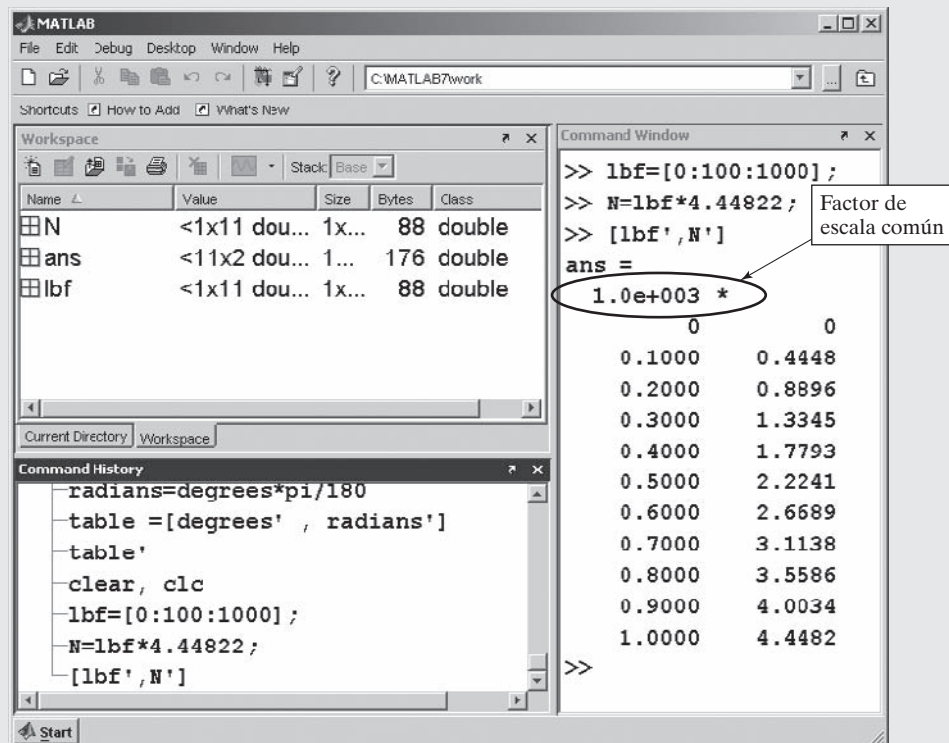
**Figura 2.8**

Mars Climate Orbiter.
(Cortesía de NASA/Jet
Propulsion Laboratory.)

4. Desarrolle una solución en MATLAB.

```
clear, clc
lbf = [0:100:1000];
N = lbf * 4.44822;
[lbf',N']
ans =
    1.0e+003 *
         0         0
    0.1000    0.4448
    0.2000    0.8896
    0.3000    1.3345
    0.4000    1.7793
    0.5000    2.2241
    0.6000    2.6689
    0.7000    3.1138
    0.8000    3.5586
    0.9000    4.0034
    1.0000    4.4482
```

Siempre es una buena idea limpiar tanto el área de trabajo como la ventana de comandos antes de comenzar un nuevo problema. Note en la ventana del área de trabajo (figura 2.9) que **lbf** y **N** son matrices de 1×11 y que **ans** (que es donde se almacena la tabla creada) es una matriz de 11×2 . La salida de los primeros dos comandos se suprimió al agregar un punto y coma al final de cada línea. Sería muy fácil crear una tabla con más entradas al cambiar el incremento a 10 o incluso a 1. Note también que necesitará multiplicar los resultados que se muestran en la tabla por 1000 para obtener las respuestas correctas. MATLAB le dice que esto es necesario directamente arriba de la tabla, donde se muestra el factor de escala común.

**Figura 2.9**

La ventana del área de trabajo MATLAB muestra las variables conforme se crean.

5. Ponga a prueba la solución.

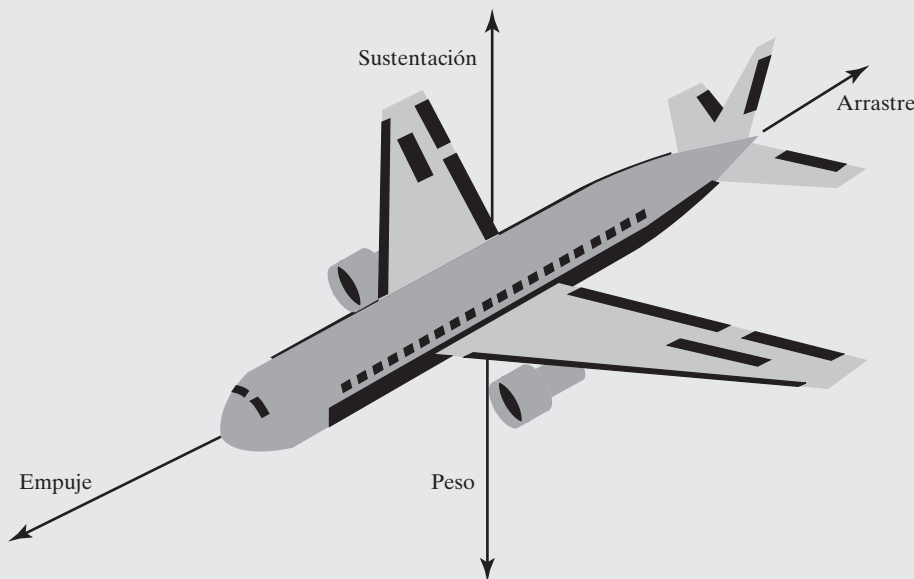
La comparación de los resultados de la solución MATLAB con la solución a mano muestra que son iguales. Una vez verificado que la solución funciona, es fácil usar el mismo algoritmo para crear otras tablas de conversión. Por ejemplo, modifique este ejemplo para crear una tabla que convierta newtons (N) a libras fuerza (lbf), con un incremento de 10 N, desde 0 N hasta 1000 N.

EJEMPLO 2.3**Cálculo del arrastre**

Una característica de rendimiento que se puede determinar en un túnel de viento es el arrastre (drag). La fricción relacionada con el arrastre en el Mars Climate Observer (producida por la atmósfera de Marte) resultó en el incendio de la nave espacial durante las correcciones de curso. El arrastre también es extremadamente importante en el diseño de aeronaves terrestres. (Véase la figura 2.10.)

El arrastre es la fuerza que se genera conforme un objeto, como un avión, se mueven a través de un fluido. Desde luego, en el caso de un túnel de viento, el aire pasa sobre un modelo estacionario, pero las ecuaciones son las mismas. El arrastre es una fuerza complicada que depende de muchos factores. Un factor es la fricción de piel, que es una función de las propiedades de la superficie de la aeronave, las propiedades del fluido en movimiento (aire en este caso) y de los patrones de flujo provocados por la forma de la aeronave (o, en el caso del Mars Climate Observer, por la nave espacial). El arrastre se puede calcular con la ecuación de arrastre

$$\text{arrastre} = C_d \frac{\rho V^2 A}{2}$$

**Figura 2.10**

El arrastre es una fuerza mecánica generada por un objeto sólido que se mueve a través de un fluido.

donde C_d = coeficiente de arrastre (drag), que se determina experimentalmente, por lo general en un túnel de viento,
 ρ = densidad del aire,
 V = velocidad de la aeronave,
 A = área de referencia (el área superficial sobre la que fluye el aire).

Aunque el coeficiente de arrastre no es una constante, se puede considerar como constante a rapideces bajas (menores a 200 mph). Suponga que los siguientes datos se midieron en un túnel de viento:

arrastre (drag)	20,000 N
ρ	$1 \times 10^{-6} \text{ kg/m}^3$
V	100 mph (necesitará convertir esto a metros por segundo)
A	1 m^2

Calcule el coeficiente de arrastre. Finalmente, use este coeficiente de arrastre determinado experimentalmente para predecir cuánto arrastre se ejercerá sobre la aeronave a rapideces desde 0 mph hasta 200 mph.

1. Establezca el problema.

Calcule el coeficiente de arrastre sobre la base de los datos recopilados en un túnel de viento. Use el coeficiente de arrastre para determinar el arrastre a varias rapideces.

2. Describa las entradas y salidas.

Entrada

Arrastre (drag)	20,000 N
Densidad del aire ρ	$1 \times 10^{-6} \text{ kg/m}^3$
Rapidez V	100 mph
Área de la superficie A	1 m^2

Salida

Coeficiente de arrastre
 Arrastre a rapideces de 0 a 200 mph

3. Desarrolle un ejemplo a mano.

Primero encuentre el coeficiente de arrastre a partir de los datos experimentales. Note que la rapidez está en millas/h y se debe cambiar a unidades consistentes con el resto de los datos (m/s). ¡Jamás se enfatizará demasiado la importancia de acarrear unidades en los cálculos de ingeniería!

$$\begin{aligned}
 C_d &= \text{arrastre} \times 2 / (\rho \times V^2 \times A) \\
 &= \frac{(20,000 \text{ N} \times 2)}{1 \times 10^{-6} \text{ kg/m}^3 \times \left(100 \text{ millas/h} \times 0.4470 \frac{\text{m/s}}{\text{millas/h}}\right)^2 \times 1 \text{ m}^2} \\
 &= 2.0019 \times 10^7
 \end{aligned}$$

Dado que un newton es igual a un kg m/s^2 , el coeficiente de arrastre es adimensional. Ahora use el coeficiente de arrastre para encontrar el arrastre a diferentes rapideces:

$$\text{arrastre} = C_d \times \rho \times V^2 \times A/2$$

Con una calculadora, encuentre el valor del arrastre con $V = 200$ mph:

$$\text{arrastre} = \frac{2.0019 \times 10^7 \times 1 \times 10^{-6} \text{ kg/m}^3 \times \left(200 \text{ millas/h} \times 0.4470 \frac{\text{m/s}}{\text{millas/h}}\right)^2 \times 1 \text{ m}^2}{2}$$

$$\text{arrastre} = 80,000 \text{ N}$$

4. Desarrolle una solución MATLAB.

```
drag = 20000;
r = 0.000001;
V = 100*0.4470;
A = 1;
cd = drag*2/(r*V^2*A)
cd =
    2.0019e+007
V = 0:20:200;
V = V*0.4470;
drag = cd*r*V.^2*A/2;
table = [V', drag']
table =
    1.0e+004 *
         0         0
    0.0009    0.0800
    0.0018    0.3200
    0.0027    0.7200
    0.0036    1.2800
    0.0045    2.0000
    0.0054    2.8800
    0.0063    3.9200
    0.0072    5.1200
    0.0080    6.4800
    0.0089    8.0000
```

Define las variables y cambia V a unidades SI.
Aquí se usó r en lugar de ρ para la densidad.

Calcula el coeficiente de arrastre.

Redefine V como matriz.
La cambia a unidades SI y calcula el arrastre.

Note que la ecuación para el arrastre, o

$$\text{drag} = \text{cd} * r * V.^2 * A/2;$$

usa el operador $.$, porque se pretende que cada valor en la matriz V sea al cuadrado, no que toda la matriz V se multiplique por sí misma. Usar sólo el operador exponenciación (^) resultaría en un mensaje de error. Desafortunadamente, es posible componer problemas en los que el uso del operador erróneo no proporciona un mensaje de error, sino que da una respuesta equivocada. Esto hace especialmente importante el paso 5 de la metodología para resolver problemas.

5. Ponga a prueba la solución.

Al comparar la solución a mano con la solución MATLAB (figura 2.11), se ve que ambas dan el mismo resultado. Una vez que se confirma que el algoritmo funciona con los datos de muestra, se pueden sustituir nuevos datos y tener la seguridad de que los resultados serán correctos. De manera ideal, los resultados también se deben comparar con los datos experimentales, para confirmar que las ecuaciones que se utilizan de manera precisa modelan el proceso físico real.

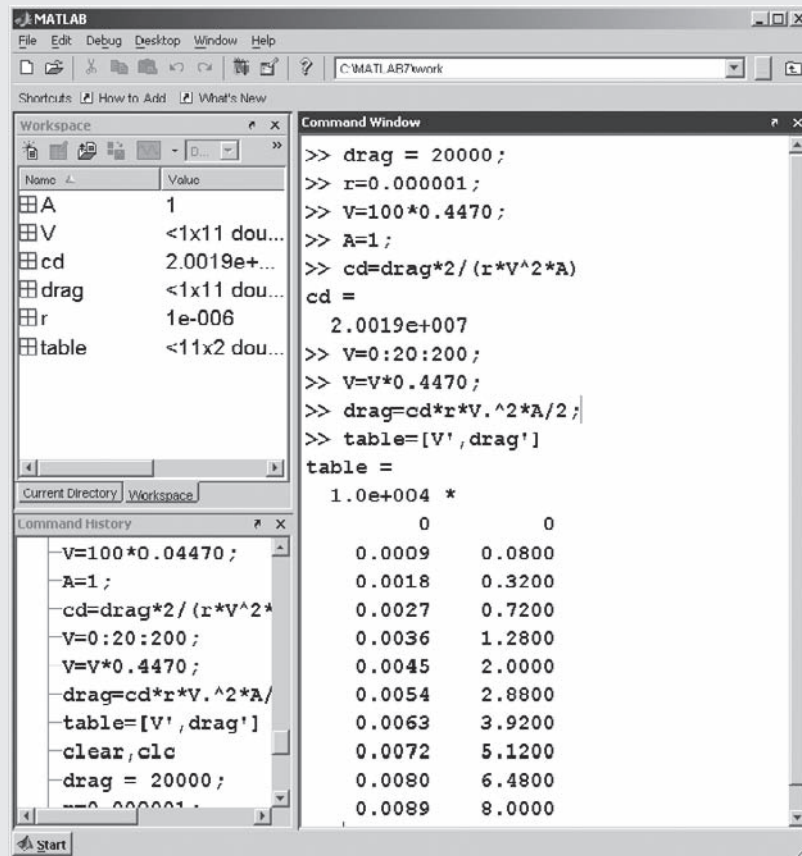


Figura 2.11

La ventana de historia de comando crea una historia de los comandos previos.

2.3.3 Despliegue de números

notación científica:

número que se representa como un valor entre uno y diez por diez a una potencia apropiada

Notación científica

Aunque es posible ingresar cualquier número en notación decimal, no siempre es la mejor forma de representar números o muy grandes o muy pequeños. Por ejemplo, un número que se usa frecuentemente en química es la constante de Avogadro, cuyo valor, a cuatro cifras significativas, es 602,200,000,000,000,000,000. De igual modo, el diámetro de un átomo de hierro es aproximadamente 140 picómetros, que es .000000000140 metros. La notación científica expresa un valor como un número entre 1 y 10, multiplicado por una potencia de 10 (el exponente). En notación científica, el número de Avogadro se convierte en 6.022×10^{23} , y el diámetro de un átomo de hierro se vuelve 1.4×10^{-10} metros. En MATLAB, los valores en notación científica se designan con una *e* entre el número decimal y el exponente. (Probablemente su calculadora usa notación similar.) Por ejemplo, usted puede tener

```
Avogadros_constant = 6.022e23 ;
Iron_diameter = 140e-12; o
Iron_diameter = 1.4e-10;
```

Es importante omitir los espacios en blanco entre el número decimal y el exponente. Por ejemplo, MATLAB interpretará

```
6.022 e23
```

como dos valores (6.022 y 10^{23}).

Sugerencia

Aunque es una convención común usar **e** para identificar una potencia de 10, los estudiantes (y el profesor) a veces confunden esta nomenclatura con la constante matemática *e*, que es igual a 2.7183. Para elevar *e* a una potencia, use la función **exp**.

Formato de despliegue

En MATLAB están disponibles algunos formatos de despliegue. No importa cuál formato de despliegue elija, MATLAB usa en sus cálculos números punto flotante de doble precisión. Exactamente cuántos dígitos se usan depende de su cálculo. Sin embargo, cambiar el formato de despliegue no cambia la precisión de sus resultados. A diferencia de algunos otros programas, MATLAB maneja los números enteros y decimales como números de punto flotante.

Cuando los elementos de una matriz se despliegan en MATLAB, los enteros siempre se imprimen sin punto decimal. No obstante, los valores con fracciones decimales se imprimen en el formato corto por defecto que muestra cuatro dígitos decimales. Por ende,

```
A = 5
```

regresa

```
A =
      5
```

pero

```
A = 5.1
```

regresa

```
A =
  5.1000
```

y

```
A = 51.1
```

regresa

```
A =
51.1000
```

MATLAB le permite especificar otros formatos que muestren dígitos significativos adicionales. Por ejemplo, para especificar que usted quiere que los valores se desplieguen en un formato decimal con 14 dígitos decimales, use el comando

```
format long
```

que cambia todos los despliegues subsecuentes. Por ende, con especificación **format long**,

```
A
```

ahora regresa

```
A =
51.10000000000000
```

Idea clave: MATLAB

no diferencia entre números enteros y de punto flotante, a menos que se invoquen funciones especiales.

Idea clave: no

importa cuál formato de despliegue se seleccione, los cálculos se realizan usando números de punto flotante con doble precisión.

Cuando el formato se especifica como **format bank** (formato banco), se despliegan dos dígitos decimales:

```
A =
    51.10
```

Usted puede regresar el formato a cuatro dígitos decimales con el comando

```
format short
```

Para verificar los resultados, usted puede recordar el valor de **A**:

```
A
A =
    51.1000
```

Cuando los números se vuelven demasiado largos o demasiado pequeños para que MATLAB los despliegue en el formato por defecto, automáticamente los expresa en notación científica. Por ejemplo, si usted ingresa la constante de Avogadro en MATLAB en notación decimal como

```
a=602000000000000000000000
```

el programa regresa

```
a =
    6.0200e+023
```

Usted puede forzar a MATLAB a desplegar *todos* los números en notación científica con **format short e** (con cuatro dígitos decimales) o **format long e** (con 14 dígitos decimales). Por ejemplo,

```
format short e
x = 10.356789
```

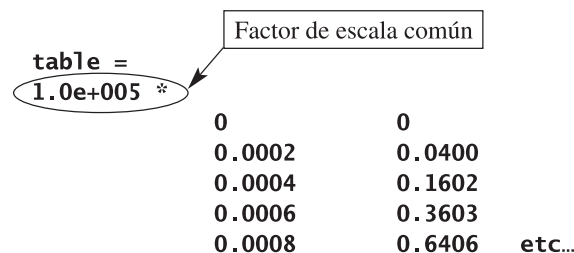
regresa

```
x =
    1.0357e+001
```

Con los formatos largo y corto se aplica un factor de escala común a toda la matriz, si los elementos se vuelven muy largos o muy cortos. Este factor de escala se imprime junto con los valores escalados. Por ejemplo, cuando la ventana de comando regresa a

```
format short
```

los resultados del ejemplo 2.3 se despliegan como



```
table =
1.0e+005 *
    0          0
    0.0002     0.0400
    0.0004     0.1602
    0.0006     0.3603
    0.0008     0.6406 etc...
```

Otro par de formatos que ocasionalmente pueden ser útiles son **format +** y **format rat**. Cuando una matriz se despliega en **format +**, los únicos caracteres que se imprimen son los

Tabla 2.2 Formatos de despliegue numérico

Comando MATLAB	Despliegue	Ejemplo
<code>format short</code>	4 dígitos decimales	3.1416
<code>format long</code>	14 dígitos decimales	3.14159265358979
<code>format short e</code>	4 dígitos decimales	3.1416e+000
<code>format long e</code>	14 dígitos decimales	3.141592653589793e+000
<code>format bank</code>	2 dígitos decimales	3.14
<code>format +</code>	+, −, espacio en blanco	+
<code>format rat</code>	forma fraccional	355/113

signos más y menos. Si un valor es positivo, se desplegará un signo más; si un valor es negativo, se desplegará un signo menos. Si un valor es cero, no se desplegará nada. Este formato permite visualizar una matriz grande en términos de sus signos:

```
format +
B = [1, -5, 0, 12; 10005, 24, -10, 4]
B =
      +- +
      ++-+
```

El comando **format rat** despliega números como números racionales (es decir: como fracciones). En consecuencia,

```
format rat
x = 0:0.1:0.5
```

regresa

```
x =
      0      1/10      1/5      3/10      2/5      1/2
```

El comando **format** también le permite controlar cuán apretadamente se espacia la información en la ventana de comando. Por defecto (**format loose**) se inserta un salto de línea entre las expresiones proporcionadas por el usuario y los resultados que regresa la computadora. El comando **format compact** remueve dichos saltos de línea. Los ejemplos en este texto usan el formato compacto para ahorrar espacio. La tabla 2.2 muestra cómo se despliega el valor de π en cada formato.

Si ninguno de estos formatos de despliegue numérico predefinidos son adecuados para usted, puede controlar líneas individuales de salida con la función **fprintf**.

número racional:
número que se puede
representar como una
fracción

2.4 CÓMO GUARDAR EL TRABAJO

Trabajar en la ventana de comandos es similar a realizar cálculos en su calculadora científica. Cuando apaga la calculadora o cuando sale del programa, su trabajo desaparece. Es posible guardar los *valores* de las variables que definió en la ventana de comando y que se enlisten en la ventana del área de trabajo, pero aunque hacerlo es útil, es más probable que usted quiera guardar la lista de comandos que generaron sus resultados. En esta sección, primero se mostrará cómo guardar y recuperar variables (los resultados de las asignaciones que hizo y los cálculos que realizó) a archivos MAT o a archivos DAT. Luego se introducirán los archivos-m script, que se crean en la ventana de edición. Los archivos-m script le permiten guardar una

lista de comandos y ejecutarlos más tarde. Descubrirá que los archivos-m script son especialmente útiles para resolver problemas de tarea en casa.

2.4.1 Cómo guardar variables

Para preservar las variables que creó en la **ventana de comandos** (verifique la lista de variables en la **ventana de área de trabajo** en el lado izquierdo de la pantalla de MATLAB) entre sesiones, debe guardar los contenidos de la **ventana de área de trabajo** a un archivo. El formato por defecto es un archivo binario llamado archivo MAT. Para guardar el área de trabajo (recuerde, éstas son sólo las variables, no la lista de comandos en la ventana de comandos) a un archivo, escriba

```
save < file_name >
```

en el prompt. Aunque **save** es un comando MATLAB, **file_name** es un nombre de archivo definido por el usuario. En este texto, los nombres definidos por el usuario se indicarán colocándolos dentro de paréntesis angulares (< >). El nombre del archivo puede ser cualquier nombre que elija, en tanto esté en conformidad con las convenciones de nomenclatura para las variables en MATLAB. De hecho, incluso ni siquiera necesita proporcionar un nombre de archivo. Si no lo hace, MATLAB nombra al archivo **matlab.mat**. También podría elegir

File → Save Workspace As

de la barra de menú, que luego lo conminará a ingresar un nombre de archivo para sus datos. Para restaurar un área de trabajo, escriba

```
load < file_name >
```

De nuevo, **load** es un comando MATLAB, pero **file_name** es el nombre de archivo definido por el usuario. Si sólo escribe **load**, MATLAB buscará el archivo **matlab.mat** por defecto.

El archivo que guarde se almacenará en el directorio actual.

Por ejemplo, escriba

```
clear, clc
```

Este comando limpiará tanto el área de trabajo como al ventana de comandos. Verifique que el área de trabajo esté vacía mediante la comprobación de la ventana del área de trabajo o al escribir

```
whos
```

Ahora defina varias variables, por ejemplo

```
a = 5;  
b = [1,2,3];  
c = [ 1, 2; 3,4];
```

Compruebe de nuevo la ventana del área de trabajo, para confirmar que las variables se almacenaron. Ahora, guarde el área de trabajo a un archivo llamado **my_example_file**:

```
save my_example_file
```

Confirme que en el directorio actual se almacenó un nuevo archivo. Si prefiere guardar el archivo a otro directorio (por ejemplo, a un disco extraíble), use el botón de navegador (véase la figura 2.2) para navegar hacia el directorio de su elección. Recuerde que, en una computadora pública, probablemente el directorio actual se purgue después de que cada usuario sale del sistema.

Ahora, limpie el área de trabajo y la ventana de comandos escribiendo

```
clear, clc
```

La ventana de área de trabajo debe estar vacía. Puede recuperar las variables perdidas y sus valores al cargar el archivo (`my_example_file.mat`) de nuevo en el área de trabajo:

```
load my_example_file
```

Recuerde: el archivo que quiera cargar debe estar en el directorio actual, o de otro modo MATLAB no podrá encontrarlo. Escriba en la ventana de comandos

```
a
```

que regresa

```
a =  
5
```

De igual modo,

```
b
```

regresa

```
b =  
1 2 3
```

y al escribir

```
c
```

regresa

```
c =  
1 2  
3 4
```

MATLAB también puede almacenar matrices individuales o listas de matrices en el directorio actual con el comando

```
save <file_name> <variable_list>
```

donde **file_name** es el nombre de archivo definido por el usuario que designa la ubicación en memoria en la que desea almacenar la información, y donde **variable_list** es la lista de variables a almacenar en el archivo. Por ejemplo,

```
save my_new_file a b
```

guardaría sólo las variables **a** y **b** en **my_new_file.mat**.

Si los datos que guardó los usará un programa distinto a MATLAB (como C o C++), el formato `.mat` no es apropiado porque los archivos `.mat` son exclusivos de MATLAB. El formato ASCII es estándar entre plataformas de computadoras y es más apropiado si necesita compartir archivos. MATLAB le permite guardar archivos como archivos ASCII al modificar el comando `save` a

```
save <file_name> <variable_list> -ascii
```

El comando `-ascii` le dice a MATLAB que almacene los datos en un formato de texto estándar de ocho dígitos. Los archivos ASCII se guardarán en un archivo `.dat` en lugar de en un archivo `.mat`; sólo asegúrese de agregar `.dat` a su nombre de archivo:

```
save my_new_file.dat a b -ascii
```

Si no agrega `.dat`, MATLAB le pondrá por defecto `.mat`.

ascii: formato de almacenamiento de datos binarios

Idea clave: cuando guarda el área de trabajo, usted sólo guarda las variables y sus valores; no guarda los comandos que ejecutó.

Si necesita más precisión, los datos se pueden almacenar en un formato de texto de 16 dígitos:

```
save file_name variable_list -ascii -double
```

También es posible delimitar los elementos (números) con tabuladores:

```
save <file_name> <variable_list> -ascii -double -tabs
```

Puede recuperar los datos del directorio actual con el comando load:

```
load <file_name>
```

Por ejemplo, para crear la matriz **z** y guardarla al archivo **data_2.dat** en formato de texto de ocho dígitos, use los comandos siguientes:

```
z = [5 3 5; 6 2 3];
save data_2.dat z -ascii
```

En conjunto, dichos comandos hacen que cada fila de la matriz **z** se escriba a una línea separada en el archivo de datos. Usted puede ver el archivo **data_2.dat** al hacer doble clic en el nombre del archivo en la ventana de directorio actual. (Véase la figura 2.12.) Acaso la forma más sencilla de recuperar datos de un archivo ASCII .dat sea ingresar el comando **load** seguido por el nombre del archivo. Esto hace que la información se lea en una matriz con el mismo nombre que el archivo de datos. Sin embargo, también es muy fácil usar el Import Wizard (asistente de importación) interactivo de MATLAB para cargar los datos. Cuando hace doble clic en el nombre del archivo de datos en el directorio actual para ver el contenido del archivo, el Import Wizard se lanzará automáticamente. Sólo siga las instrucciones para cargar los datos en el área de trabajo con el mismo nombre que el archivo de datos. Puede usar esta misma técnica para importar datos de otros programas, incluso hojas de cálculo Excel, o puede seleccionar **File** → **Import data...** de la barra de menú.

2.4.2 Archivos-m script

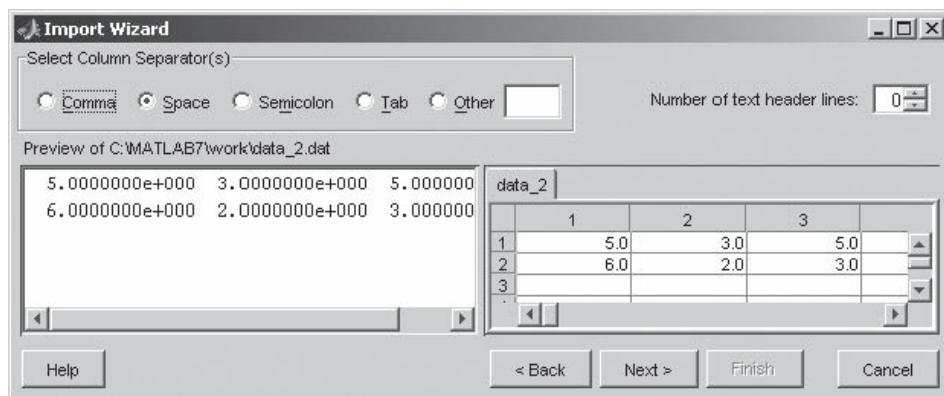
Además de proporcionar un ambiente computacional interactivo (al usar la ventana de comandos como una memoria de trabajo auxiliar), MATLAB contiene un poderoso lenguaje de programación. Como programador, puede crear y guardar código en archivos llamados archivos-m. Un archivo-m es un archivo de texto ASCII similar a los archivos de código fuente de C o FORTRAN. Se puede crear y editar con el editor/debugger (depurador) de archivo-m de MATLAB (la ventana de edición que se analizó en la sección 2.2.7), o puede usar otro editor de texto de su elección. Para abrir la ventana de edición, seleccione

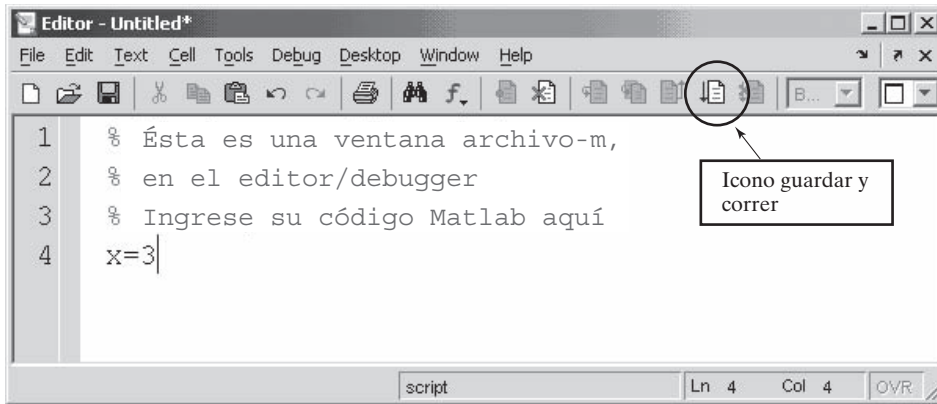
File → **New** → **M-file**

de la barra de menú de MATLAB. En la figura 2.13 se muestra la ventana de edición de MATLAB.

Figura 2.12

Hacer doble clic en el nombre de archivo en el directorio de comandos lanza el Import Wizard.



**Figura 2.13**

Ventana de edición de MATLAB, también llamado editor/debugger.

Si usted elige un editor de texto diferente, asegúrese de que los archivos que guarde sean archivos ASCII. Notepad es un ejemplo de editor de texto que por defecto origina una estructura de archivo ASCII. Otros procesadores de palabra, como WordPerfect o Word, requerirán que usted especifique la estructura ASCII cuando guarde el archivo. Esos programas dan por defecto estructuras de archivo propietario que no se someten a ASCII, y pueden producir algunos resultados inesperados si intenta usar código escrito en ellos si no especifica que los archivos se guarden en formato ASCII.

Cuando se guarda un archivo-m, éste se almacena en el directorio actual. Será necesario que nombre su archivo con un nombre variable MATLAB válido, esto es, un nombre que comience con una letra y contenga sólo letras, números y el guión bajo (_). No se permiten los espacios. (Véase la sección 2.3.1.)

Existen dos tipos de archivos-m, llamados scripts y funciones. Un archivo-m script es simplemente una lista de enunciados MATLAB que se guardan en un archivo (por lo general, con una extensión de archivo .m). El script puede usar cualesquiera variables que se hayan definido en el área de trabajo, y cualesquiera variables que se creen en el script se agregarán al área de trabajo cuando el script termine. Puede ejecutar un script creado en la ventana de edición MATLAB al seleccionar el icono Save and Run (guardar y correr) de la barra de menú, como se muestra en la figura 2.13. De manera alternativa, puede ejecutar un script al escribir un nombre de archivo o al usar el comando run de la ventana de comandos.

Suponga que creó un archivo script llamado myscript.m. Puede correr el script desde la ventana de edición o usar una de tres formas de ejecutar el script desde la ventana de comandos. (Véase la tabla 2.3.) Las tres técnicas son equivalentes. Usted elige cuál usar.

Puede descubrir cuáles son los archivos-m y MAT en el directorio actual al escribir

```
what
```

en la ventana de comandos. También puede navegar en el directorio actual al buscar en la ventana de directorio actual.

archivo-m: lista de comandos MATLAB almacenados en un archivo separado

Idea clave: los dos tipos de archivos-m son scripts y funciones.

Tabla 2.3 Enfoques para ejecutar un archivo-m script desde la ventana de comandos

Comando MATLAB	Comentarios
<code>myscript</code>	Escriba el nombre de archivo. Se supone la extensión de archivo .m
<code>run myscript</code>	Use el comando run con el nombre de archivo.
<code>run('myscript')</code>	Use la forma funcional del comando run

Usar archivos-m script le permite trabajar en un proyecto y guardar la lista de comandos para uso futuro. Puesto que más adelante usará estos archivos, es buena idea salpicarlos libremente con comentarios. El operador comentario en MATLAB es el signo de porcentaje, como en

```
% Esto es un comentario
```

MATLAB no ejecutará código alguno en una línea comentada.

También puede agregar comentarios después de un comando, pero en la misma línea:

```
a = 5           %La variable a se define como 5
```

El código MATLAB que se podría ingresar en un archivo-m y utilizarse para resolver el ejemplo 2.3 es como sigue:

```
clear, clc
% Archivo-m script para encontrar drag (arrastre)
% Primero defina las variables
drag = 20000;           %Defina drag en newtons
r = 0.000001;           %Defina densidad del aire en kg/m^3
V = 100*0.4470;         %Defina rapidez en m/s
A = 1;                  %Defina área en m^2
% Calcula coeficiente de arrastre
cd = drag *2/(r*V^2*A)
% Encuentra el arrastre para varias rapidezces
V = 0:20:200;           %Redefine rapidez
V = V*.4470             %Cambia rapidez a m/s
drag = cd*r*V.^2*A/2;    %Calcula arrastre
table = [V',drag']       %Crea una tabla de resultados
```

Este código se podría correr o desde el archivo-m o desde la ventana de comandos. En cualquier caso, los resultados aparecerán en la ventana de comandos, y las variables se almacenarán en el área de trabajo.

Sugerencia

Puede ejecutar una porción de un archivo-m al resaltar una sección y luego hacer clic derecho y seleccionar **Evaluate Section** (evaluar sección). También desde este menú puede comentar o “descomentar” secciones completas de código; hacerlo así es útil cuando crea programas mientras todavía depura su trabajo.

El ejemplo final de este capítulo usa un archivo-m script para encontrar la rapidez y aceleración que puede alcanzar una nave espacial al salir del sistema solar.

EJEMPLO 2.4

Creación de un archivo-m para calcular la aceleración de una nave espacial

En ausencia de arrastre (drag), los requisitos de potencia de propulsión para una nave espacial se determinan de manera bastante simple. Recuerde de la física básica que

$$F = ma$$

En otras palabras, fuerza (F) es igual a masa (m) por aceleración (a). El trabajo (W) es fuerza por distancia (d), y dado que la potencia (P) es trabajo por unidad de tiempo, la potencia se convierte en fuerza por velocidad (v):

$$W = Fd$$

$$P = \frac{W}{t} = F \times \frac{d}{t} = F \times v = m \times a \times v$$

Esto significa que los requerimientos de potencia para la nave espacial dependen de su masa, de qué tan rápido marche y de qué tan rápidamente necesite acelerar o frenar. Si no se aplica potencia, la nave espacial simplemente sigue viajando a su velocidad actual. En tanto no se quiera hacer algo rápidamente, las correcciones de curso se pueden realizar con muy poca potencia. Desde luego, la mayoría de los requerimientos de potencia para la nave espacial no se relacionan con la navegación. La potencia se requiere para comunicación, para mantenimiento y para experimentos y observaciones científicos.

Las naves espaciales *Voyager 1* y *2* exploraron el sistema solar exterior durante el último cuarto del siglo xx. (Véase la figura 2.14.) El *Voyager 1* se encontró con Júpiter y Saturno; el *Voyager 2* no sólo encontró a Júpiter y Saturno sino que continuó hacia Urano y Neptuno. El programa Voyager fue enormemente exitoso y la nave espacial *Voyager* continúa recopilando información conforme se aleja del sistema solar. Se espera que los generadores de potencia (reactores nucleares de bajo nivel) en cada nave espacial funcionen hasta al menos el año 2020. La fuente de poder es una muestra de plutonio 238 que, conforme decae, genera calor que se usa para producir electricidad. Durante el lanzamiento de cada nave espacial, su generador produjo alrededor de 470 watts de potencia. Puesto que el plutonio decae, la producción de potencia disminuyó a aproximadamente 335 watts en 1997, casi 20 años después del lanzamiento. Esta potencia se usa para operar el paquete científico, pero si se dirigiese a propulsión, ¿cuánta aceleración produciría en la nave espacial? En la actualidad, el *Voyager 1* viaja con una rapidez de 3.50 AU/año (AU es unidad astronómica), y el *Voyager 2* viaja a 3.15 AU/año. Cada nave espacial pesa 721.9 kg.

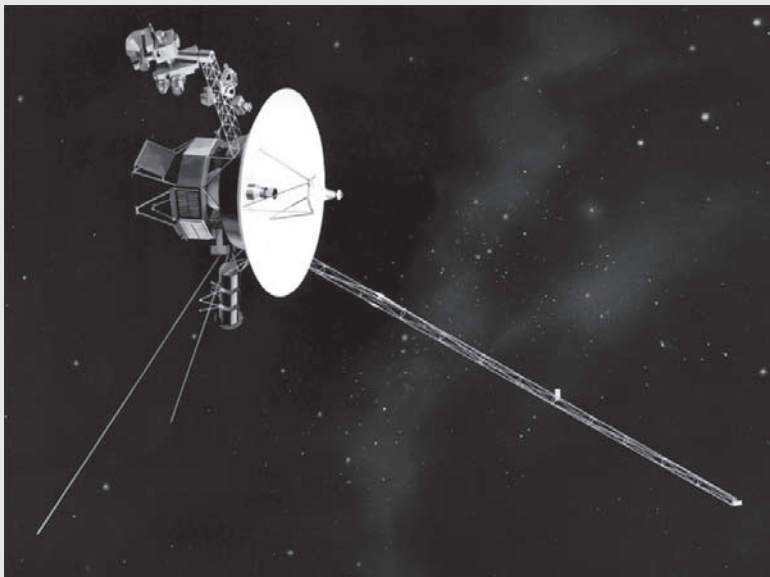


Figura 2.14

Las naves espaciales *Voyager 1* y *Voyager 2* se lanzaron en 1977 y ya abandonaron el sistema solar. (Cortesía de NASA/Jet Propulsion Laboratory.)

1. Establezca el problema.
Encontrar la aceleración que es posible con la salida de potencia de los generadores de potencia de la nave espacial.
2. Describa las entradas y salidas.

Entrada

Masa = 721.9 kg
 Potencia = 335 watts = 335 J/s
 Rapidez = 3.50 AU/año (Voyager 1)
 Rapidez = 3.15 AU/año (Voyager 2)

Salida

Aceleración de cada nave espacial, en m/s/s

3. Desarrolle un ejemplo a mano.
Se sabe que

$$P = m \times a \times v$$

que se puede reordenar para obtener

$$a = P/(m \times v)$$

La parte más difícil de este cálculo será conservar las unidades correctas. Primero, cambie la rapidez a m/s. Para el *Voyager 1*,

$$v = 3.50 \frac{\text{AU}}{\text{año}} \times \frac{150 \times 10^9 \text{ m}}{\text{AU}} \times \frac{\text{año}}{365 \text{ días}} \times \frac{\text{día}}{24 \text{ horas}} \times \frac{\text{hora}}{3600 \text{ s}} = 16,650 \frac{\text{m}}{\text{s}}$$

Luego se calcula la aceleración:

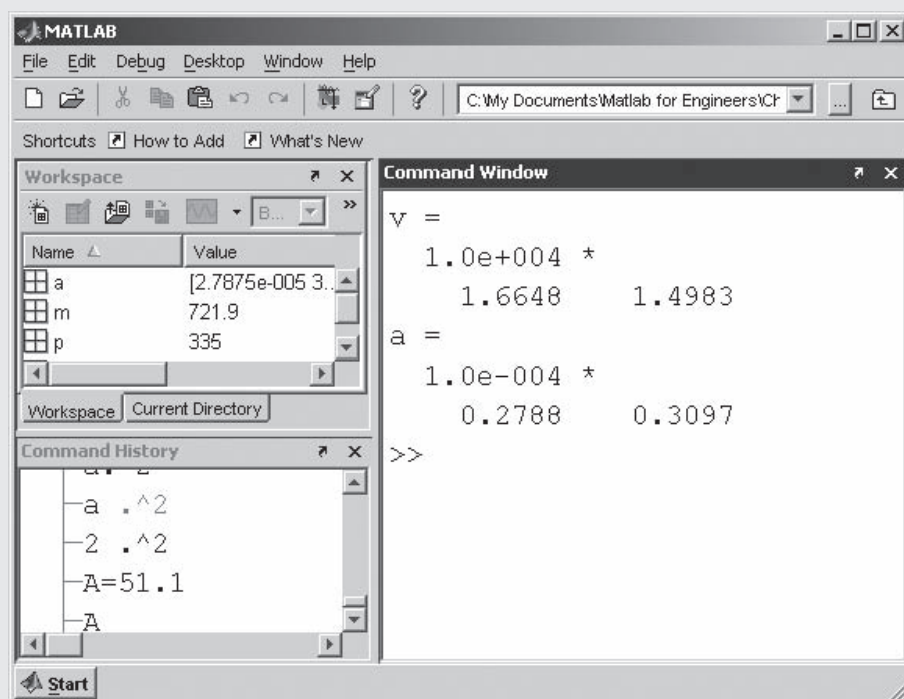
$$a = \frac{335 \frac{\text{J}}{\text{s}} \times 1 \frac{\text{kg} \times \text{m}^2}{\text{s}^2 \text{ J}}}{721.9 \text{ kg} \times 16,650 \frac{\text{m}}{\text{s}}} = 2.7 \times 10^{-5} \frac{\text{m}}{\text{s}^2}$$

4. Desarrolle una solución MATLAB.

```

clear, clc
%Ejemplo 2.4
%Encuentre la posible aceleración de las naves espaciales
%Voyager 1 y Voyager 2 usando el generador de potencia
%a bordo
format short
m=721.9;      %masa en kg
p=335;        % potencia en watts
v=[3.5 3.15]; % rapidez en AU/año
%Cambia la rapidez a m/s
v=v*150e9/365/24/3600
%Calcula la aceleración
a=p./(m.*v)
  
```

Los resultados se imprimen en la ventana de comandos, como se muestra en la figura 2.15.

**Figura 2.15**

Los resultados de la ejecución de un archivo-m se imprimen en la ventana de comandos. Las variables que se crearon se reflejan en el área de trabajo y el archivo-m se cita en la ventana de directorio actual. Los comandos emitidos en el archivo-m no se reflejan en la historia de comandos.

5. Ponga a prueba la solución.

Compare los resultados MATLAB con los resultados del ejemplo a mano. Note que la rapidez y la aceleración calculadas en el ejemplo a mano coinciden con la solución MATLAB para el *Voyager 1*. La aceleración parece muy pequeña, pero al aplicarse sobre periodos de semanas o meses, tal aceleración puede lograr cambios de velocidad significativos. Por ejemplo, una aceleración constante de $2.8 \times 10^{-5} \text{ m/s}^2$ resulta en un cambio de velocidad de aproximadamente 72 m/s durante un mes:

$$\begin{aligned}
 &2.8 \times 10^{-5} \text{ m/s}^2 \times 3600 \text{ s/hora} \\
 &\times 24 \text{ horas/día} \times 30 \text{ días/mes} = 72.3 \text{ m/s}
 \end{aligned}$$

Ahora que tiene un programa MATLAB que funciona, puede usarlo como el punto de partida para otros cálculos más complicados.

RESUMEN

En este capítulo se introdujo la estructura MATLAB básica. El ambiente MATLAB incluye múltiples ventanas, cuatro de las cuales se abren en la vista por defecto:

- Ventana de comandos (command window).
- Ventana de historia de comandos (command history).
- Ventana de área de trabajo (workspace).
- Ventana de directorio actual (current directory).

Además, las ventanas

- de documentos.
- de gráficas.
- de edición.

se abren conforme se requieran durante una sesión MATLAB.

Las variables que se definen en MATLAB siguen las convenciones de nomenclatura computacional comunes:

- Los nombres deben comenzar con una letra.
- Letras, números y guión bajo son los únicos caracteres permitidos.
- Los nombres son sensibles a mayúsculas y minúsculas.
- Los nombres pueden tener cualquier longitud, aunque MATLAB sólo usa los primeros 63 caracteres.
- Algunas palabras clave se reservan para MATLAB y no se pueden usar como nombres de variables.
- MATLAB permite al usuario reasignar nombres de función como nombres de variable, aunque no es aconsejable hacerlo.

La unidad computacional básica en MATLAB es la matriz. Las matrices pueden ser.

- Escalares (matriz 1×1).
- Vectores (matrices $1 \times n$ o $n \times 1$, o una fila o una columna).
- Arreglos bidimensionales ($m \times n$ o $n \times m$).
- Arreglos multidimensionales.

Con frecuencia, las matrices almacenan información numérica, aunque también pueden almacenar otro tipo de información. Los datos se pueden ingresar en una matriz de forma manual o se pueden recuperar de archivos de datos almacenados. Cuando se ingresa manualmente, una matriz se encierra en corchetes, los elementos en una fila se separan mediante comas o espacios, y una nueva fila se indica con un punto y coma:

```
a = [1 2 3 4; 5 6 7 8]
```

Las matrices igualmente espaciadas se pueden generar con el operador dos puntos. Por ende, el comando

```
b = 0:2:10
```

crea una matriz que comienza en 0, termina en 10 y tiene un incremento de 2. Las funciones **linspace** y **logspace** se pueden usar para generar una matriz de longitud especificada a partir de valores de inicio y fin dados, espaciados lineal o logarítmicamente. La función **help** o el menú Help MATLAB se pueden usar para determinar la sintaxis apropiada para estas y otras funciones.

MATLAB sigue el orden algebraico estándar de las operaciones. Los operadores que soporta MATLAB se mencionan en la sección “Resumen MATLAB” de este capítulo.

MATLAB soporta notación estándar (decimal) y científica. También soporta algunas diferentes opciones de despliegue, que también se describen en la sección “Resumen MATLAB”. Sin importar cómo se desplieguen los valores, se almacenan como números de punto flotante de doble precisión.

Las colecciones de comandos MATLAB se pueden guardar en archivos-m script. Las variables MATLAB se pueden guardar o importar de archivos .MAT o .DAT. El formato .MAT es propio de MATLAB y se usa porque almacena datos de manera más eficiente que otros formatos de archivo. El formato .DAT emplea el formato ASCII estándar y se usa cuando los datos creados en MATLAB se compartirán con otros programas.

El siguiente resumen MATLAB cita todos los caracteres especiales, comandos y funciones que se definieron en este capítulo:

RESUMEN MATLAB

Caracteres especiales	
[]	forma matrices
()	se usa en enunciados para agrupar operaciones
,	se usa con un nombre de matriz para identificar elementos específicos
;	separa subíndices o elementos de matriz
:	separa filas en una definición de matriz
:	suprime salida cuando se usa en comandos
:	se usa para generar matrices
:	indica todas las filas o todas las columnas
=	operador asignación: asigna un valor a una ubicación de memoria; no es lo mismo que una igualdad
%	indica un comentario en un archivo-m
+	suma escalar y de arreglo
-	resta escalar y de arreglo
*	multiplicación escalar y multiplicación en álgebra matricial
.*	multiplicación de arreglo (multiplicación punto o punto estrella)
/	división escalar y división en álgebra matricial
./	división de arreglo (división punto o punto diagonal)
^	exponenciación escalar y exponenciación de matriz en álgebra matricial
.^	exponenciación de arreglo (potencia punto o punto caret)
Comandos y funciones	
ans	nombre de variable por defecto para resultados de cálculos MATLAB
ascii	indica que los datos se deben guardar en formato ASCII estándar
clc	limpia la ventana de comandos
clear	limpia el área de trabajo
exit	termina MATLAB
format +	establece formato sólo a signos más y menos
format compact	establece formato a forma compacta
format long	establece formato a 14 lugares decimales
format long e	establece formato a notación científica con 14 lugares decimales
format loose	establece formato a la forma no compacta por defecto
format short	establece formato al defecto, 4 lugares decimales
format short e	establece formato a notación científica con 4 lugares decimales
format rat	establece formato a despliegue racional (fraccional)
help	invoca la utilidad de ayuda
linspace	función vector linealmente espaciado
load	carga matrices desde un archivo
logspace	función vector logarítmicamente espaciado
pi	aproximación numérica del valor de π
quit	termina MATLAB
save	guarda variables en un archivo
who	lista variables en memoria
whos	lista variables y sus tamaños

archivo-m
área de trabajo
argumentos
arreglo
ASCII
asignación
botón de inicio
directorio actual

editor de arreglo
escalar
función
historia de comandos
matriz
notación científica
operador
prompt (incitador)

script
transponer
vector
ventana de comandos
ventana de documentos
ventana de edición
ventana de gráficas

TÉRMINOS CLAVE

PROBLEMAS

Inicio

2.1 Prediga el resultado de los siguientes cálculos MATLAB:

$1 + 3/4$
 $5 * 6 * 4/2$
 $5/2 * 6 * 4$
 $5^2 * 3$
 $5^(2 * 3)$
 $1 + 3 + 5/5 + 3 + 1$
 $(1 + 3 + 5)/(5 + 3 + 1)$

Verifique sus resultados al ingresar los cálculos en la ventana de comandos.

Uso de variables

2.2 Identifique cuál de cada uno de los siguientes pares es un nombre de variable MATLAB legítimo:

fred	fred!
book_1	book-1
2ndplace	Second_Place
#1	No_1
vel_5	vel.5
tan	while

Pruebe sus respuestas con el uso de **isvarname**; por ejemplo,

isvarname fred

Recuerde: **isvarname** regresa un 1 si el nombre es válido y 0 si no lo es. Aunque es posible reasignar un nombre de función como nombre de variable, hacerlo no es una buena idea. Use **which** para verificar si los nombres precedentes son nombres de funciones; por ejemplo,

which sin

¿En qué caso MATLAB le diría que **sin** es un nombre de variable, no un nombre de función?

Operaciones escalares y orden de operaciones

2.3 Cree código MATLAB para realizar los cálculos siguientes:

5^2
 $\frac{5 + 3}{5 \cdot 6}$
 $\sqrt{4 + 6^3}$ (Sugerencia: una raíz cuadrada es lo mismo que una potencia $1/2$.)
 $9\frac{6}{12} + 7 \cdot 5^{3+2}$
 $1 + 5 \cdot 3/6^2 + 2^{2-4} \cdot 1/5.5$

Compruebe su código al ingresarlo en MATLAB y mediante la realización de los cálculos en su calculadora científica.

- 2.4 (a) El área de un círculo es πr^2 . Defina r como 5 y luego encuentre en MATLAB el área de un círculo.
- (b) El área superficial de una esfera es $4\pi r^2$. Encuentre el área superficial de una esfera con un radio de 10 pies.
- (c) El volumen de una esfera es $\frac{4}{3}\pi r^3$. Encuentre el volumen de una esfera con un radio de 2 pies.
- 2.5 (a) El área de un cuadrado es la longitud de arista al cuadrado. ($A = \text{arista}^2$.) Defina la longitud de arista como 5 y luego encuentre en MATLAB el área de un cuadrado.
- (b) El área superficial de un cubo es 6 veces la longitud de arista al cuadrado. ($AS = 6 \times \text{arista}^2$.) Encuentre el área superficial de un cubo con longitud de arista 10.
- (c) El volumen de un cubo es la longitud de arista al cubo. ($V = \text{arista}^3$.) Encuentre el volumen de un cubo con longitud de arista 12.

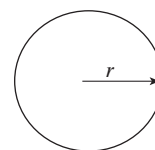


Figura P2.4(a)

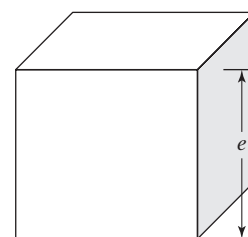
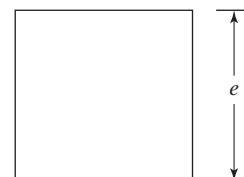


Figura P2.5(a-c)

Operaciones de arreglos

- 2.6 (a) El volumen de un cilindro es $\pi r^2 h$. Defina r como 3 y h como la matriz

$$h = [1, 5, 12]$$

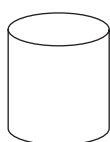
Encuentre el volumen de los cilindros.

- (b) El área de un triángulo es $\frac{1}{2}$ la longitud de la base del triángulo, por la altura del triángulo. Defina la base como la matriz

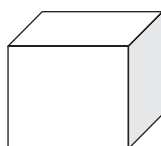
$$b = [2, 4, 6]$$

y la altura h como 12, y encuentre el área de los triángulos.

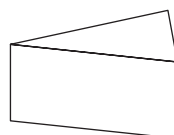
- (c) El volumen de cualquier prisma recto es el área de la base del prisma, por la dimensión vertical del prisma. La base del prisma puede ser cualquier forma, por ejemplo,



Base es
un círculo



Base es
un rectángulo



Base es
un triángulo

Figura P2.6(c)

Encuentre el volumen de los prismas creados a partir de los triángulos de la parte (b). Suponga que la dimensión vertical de estos prismas es 6.

- 2.7 (a) Cree un vector igualmente espaciado de valores desde 1 hasta 20 en incrementos de 1.
- (b) Cree un vector con valores desde cero hasta 2π en incrementos de $\pi/10$.
- (c) Cree un vector que contenga 15 valores, igualmente espaciados entre 4 y 20. (*Sugerencia:* use el comando **linspace**. Si no puede recordar la sintaxis, escriba **help linspace**.)
- (d) Cree un vector que contenga 10 valores logarítmicamente espaciados entre 10 y 1000. (*Sugerencia:* use el comando **logspace**.)
- 2.8 (a) Cree una tabla de conversión de pies a metros. Comience la columna pies en 0, incremente en 1 y termine en 10 pies. (Busque el factor de conversión en un manual o en línea.)

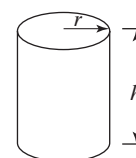


Figura P2.6(a)

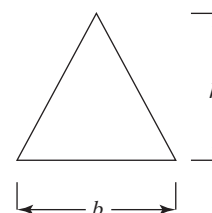


Figura P2.6(b)

- (b) Cree una tabla de conversiones de radianes a grados. Comience la columna radianes en 0 e incremente por 0.1π radián, hasta π radianes. (Busque el factor de conversión en un manual o en línea.)
- (c) Cree una tabla de conversiones de mi/h a pies/s. Comience la columna mi/h en 0 y termine en 100 mi/h. Imprima 15 valores en su tabla. (Busque el factor de conversión en un manual o en línea.)
- (d) La acidez de las soluciones, por lo general, se mide en términos de pH. El pH de una solución se define como $-\log_{10}$ de la concentración de iones hidruro. Cree una tabla de conversiones de concentración de ión hidruro a pH, logarítmicamente espaciada, de .001 a .1 mol/litro con 10 valores. Si supone que la concentración de iones hidruro se nombra **H_conc**, la sintaxis para calcular el logaritmo de la concentración es

log10(H_conc)

- 2.9** La ecuación general para la distancia que recorre un cuerpo en caída libre (ignorando la fricción del aire) es

$$d = \frac{1}{2}gt^2$$

Suponga que $g = 9.8 \text{ m/s}^2$. Genere una tabla de tiempo contra distancia recorrida para valores de tiempo de 0 a 100 segundos. Elija un incremento adecuado para su variable tiempo. (*Sugerencia:* tenga cuidado de usar los operadores correctos; $\wedge$ es una operación arreglo!)

- 2.10** La ley de Newton de la gravitación universal dice que la fuerza ejercida por una partícula sobre otra es

$$F = G \frac{m_1 m_2}{r^2}$$

donde la constante de gravitación universal G tiene el valor experimental de

$$G = 6.673 \times 10^{-11} \text{ N m}^2/\text{kg}^2$$

La masa de cada partícula es m_1 y m_2 , respectivamente, y r es la distancia entre las dos partículas. Use la ley de Newton de gravitación universal para encontrar la fuerza que ejerce la Tierra sobre la Luna, si supone que

la masa de la Tierra es aproximadamente $6 \times 10^{24} \text{ kg}$,
 la masa de la Luna es aproximadamente $7.4 \times 10^{22} \text{ kg}$, y
 la Tierra y la Luna están separadas una distancia promedio de $3.9 \times 10^8 \text{ m}$.

- 2.11** Se sabe que la Tierra y la Luna no siempre están separadas la misma distancia. Encuentre la fuerza que la Luna ejerce sobre la Tierra para 10 distancias entre $3.8 \times 10^8 \text{ m}$ y $4.0 \times 10^8 \text{ m}$.

Despliegue de números

- 2.12** Cree una matriz **a** igual a $[-1/3, 0, 1/3, 2/3]$ y use cada una de las opciones de formato interno para desplegar los resultados:

formato corto (que es por defecto)
 formato largo
 formato bank
 formato corto e
 formato largo e
 formato +
 formato rat

Cómo guardar trabajo en archivos

- 2.13**
- Cree una matriz llamada `G_a_R` compuesta de dos columnas, una que represente grados y la otra que represente el valor correspondiente en radianes. Para este ejercicio es válido cualquier conjunto de valores.
 - Guarde la matriz a un archivo llamado `degrees.dat`.
 - Una vez guardado el archivo, limpie su área de trabajo y luego cargue los datos del archivo de vuelta a MATLAB.
- 2.14**
- Cree un archivo-m script y úselo para hacer los problemas de tarea en este capítulo. Su archivo debe incluir comentarios adecuados para identificar cada problema y describir su proceso de cálculo. No olvide incluir su nombre, la fecha y cualquier otra información que solicite su profesor.